https://pubsonline.informs.org/journal/educ

TUTORIALS IN OPERATIONS RESEARCH

Vol. 00, No. 0, Xxxxx 0000, pp. 000–000

Submitted to *TutORials in Operations Research*

ISSN 0000-0000, EISSN 0000-0000

Self-Adapting Approximations of Markov Decision Processes: A Guided Tour

Andre Augusto Cire
Rotman School of Management, University of Toronto Scarborough, Toronto, ON M5S 1A1, Canada, andre.cire@rotman.utoronto.ca

Selvaprabu Nadarajah
Information and Decision Sciences, University of Illinois Chicago, Chicago, IL 60607, USA, selvan@uic.edu

Parshan Pakiman
School of Management, University at Buffalo, State University of New York, Buffalo, NY 14260, USA, parshanp@buffalo.edu

Negar Soheili
Information and Decision Sciences, University of Illinois Chicago, Chicago, IL 60607, USA, nazad@uic.edu

Abstract. Sequential decision making under uncertainty arises across business, engineering, and science. Markov Decision Processes (MDPs) offer a rich modeling language, but translating models into implementable policies requires confronting challenging state and action spaces and non-convex optimization landscapes. Approximations are essential, and making them accessible and performant without algorithmic expertise allows users to focus on formulating MDPs that capture real-world features rather than simplifying their models to facilitate solution. Obtaining a policy involves a three-stage COR cycle: **C**onstructing a parametrized model that approximates the MDP, **O**ptimizing for the parameters of this model, and **R**efining the model by learning from the solution. These stages involve design choices that burden the user, leading them to simplify or skip stages.

This tutorial provides an overivew of methods that reduce the user burden using COR as its guide. **C**onstruction ranges from lightweight extensible models to automated structural reformulations. **O**ptimization leverages mathematical programming and first-order methods that have matured in theory and off-the-shelf software. **R**efinement performs feature expansion and approximation tightening, feeding back into the construct stage. When a combination of techniques across COR stages results in both accessibility and performance, we refer to it as a self-adapting approximation. Such an approximation adapts to problem structure, instance data, or solution progress. We guide readers through these ideas across weakly coupled MDPs and general continuous state-action MDPs, providing generalizable foundations and a code base to get started on a simple example. The tutorial concludes by outlining the potential for foundation models and quantum computing to further impact this area.

Key words: Markov decision processes, sequential decision making, reinforcement learning, approximate dynamic programming, approximate linear programming, weakly coupled MDPs, mathematical programming, first-order methods, Lagrangian relaxation, self-adapting approximations, random features.

History: June 15, 2026 (current version); April 30, 2026 (initial version)

1. Introduction

Sequential decision making under uncertainty arises across business, engineering, and science. An energy company must decide when to invest in generation and storage assets as prices and demand evolve, a problem with the timing characteristics of a Bermudan option (an option exercisable

only on a fixed set of dates). A retailer must decide how much perishable inventory to order each period while facing uncertain demand, partial backlogging, and lead times. A telecommunications firm must decide which assets to maintain each week across a network where resources are shared and maintenance of certain asset pairs cannot occur simultaneously. In each of these applications, decisions are made sequentially, information is revealed over time, and decisions made today affect what is feasible tomorrow.

Markov Decision Processes (MDPs) offer a rich modeling language for these problems. An MDP captures the essential elements: states that summarize what is known, actions that represent available decisions, transitions that describe how the system evolves under uncertainty, and rewards that quantify objectives. Nonetheless, translating an MDP model into an implementable policy requires confronting challenging state and action spaces and non-convex optimization landscapes. For instance, the maintenance problem above with 40 assets tracked over 14 weeks involves a state space of approximately 10^{12} and up to 2^{40} possible actions per period. Exact solution methods cannot handle problems at this scale, and practitioners must turn to approximations.

Approximations are essential to obtain policies, but there is a gap at present between what a user can model and what they can solve. An approximation that is easy to use but yields poor policies does not close this gap, nor does one that performs well but requires expert effort to configure. Closing the gap requires approximations that are both accessible and performant without algorithmic expertise, allowing users to focus on formulating MDPs that capture real-world features rather than simplifying their models to facilitate solution approaches.

This tutorial focuses on alleviating the solution burden to bridge the gap between modeling and tractability. To this end, our approach conceptualizes policy extraction from an approximate MDP as a process governed by the so-called COR cycle, a three-stage process of **C**onstructing, **O**ptimizing, and **R**efining. In the **construct** stage, the user creates a parametrized model that approximates the MDP and possibly pre-processes it to obtain a better reformulation. Two fundamental parametric models exist: value function approximation (VFA), which parametrizes the value function using basis functions (also known as features in machine learning) and implies a policy through optimization, and policy approximation, which parametrizes the policy–either directly or through basis functions–and tunes its parameters from sampled trajectories rather than through an optimization over the value function, as in policy-gradient methods such as reinforce[23, 34]. In either model, these basis functions can take many forms, ranging from classical functional families such as polynomials to random features and network architectures.

In both types of parametric models, the construct stage defines an approximation architecture, the parameters to be determined, and an optimization model possibly reformulated to facilitate the second stage. In the **optimize** stage, the user solves for these parameters, leveraging mathematical programming or first-order methods. In the **refine** stage, the user evaluates the quality of the resulting approximation and improves it through feature expansion or approximation tightening, feeding the improved model back into the construct stage for the next iteration.

Each stage involves design decisions that burden the user, and these stages interact in ways that make skipping one costly. Consider the inventory control problem discussed at the opening. The user must design basis functions in the construct stage, and if the resulting optimization problem proves difficult, they face a dilemma: scale back the basis functions, which can degrade policy performance, or simplify the MDP itself, perhaps removing lead times or assuming away partial backlogging, which may yield a tractable optimization and good policies but for the wrong problem. Either way, the policy moves away from what is desired for technical rather than modeling reasons. Contrast this with the typical experience of using a commercial solver such as Gurobi, where users focus almost entirely on modeling a real-world problem as a mixed-integer program (MIP) and trust the solver to handle solution complexity. Not surprisingly, MIP solvers are pervasive in math, engineering, and business classrooms. MDP approximations are not yet at this level – modeling and solving remain entangled. Closing this gap, so that users can model freely and trust the approximation machinery to absorb solution complexity, is the central aspiration of this tutorial, and recent progress along this direction is what we highlight.

The success of modern MIP solvers offers a useful analogy for the structure of the COR cycle. Solvers like Gurobi do not simply apply a branch-and-bound algorithm to the user's formulation. They presolve to tighten the formulation, dynamically generate cuts during the solve, and apply heuristics such as feasibility pump to find good solutions early, which then informs downstream algorithmic decisions. These stages, analogous to construct, optimize, and refine, are automated and adaptive, and their combined effect is what makes MIP solvers so accessible and performant. The popularity of generative artificial intelligence also has this dual benefit when useful.

Recent advances in MDP approximations pursue the same vision: automating key decisions within one or more stages of the COR cycle rather than requiring the user to make all design decisions. The resulting collection of techniques is powerful because it automatically adapts to aspects such as problem structure, instance data, and solution progress. When engaging one or more stages in this way yields both accessibility and performance, we term the result a *self-adapting*

approximation. This tutorial focuses on two complementary settings where this paradigm is particularly impactful. We begin with weakly coupled MDPs, where problem structure can be explicitly leveraged through decomposition and coordination mechanisms, providing a structured testbed for self-adapting ideas. We then move to continuous-state MDPs, where such structure is less explicit and approximations must operate in more general function spaces. This progression highlights how self-adapting approximations evolve from structured environments to increasingly general settings, while maintaining their ability to balance modeling fidelity and computational tractability.

A natural question is how this differs from end-to-end deep reinforcement learning (deep RL), which trains a neural-network policy or value function directly from sampled transitions [34]. Deep RL is attractive when the environment is unstructured or available only through a simulator; the self-adapting approach we describe instead exploits a known model and, especially, its constraints, and in return provides the optimality and feasibility certificates that black-box policies lack. It is therefore most beneficial when the MDP carries exploitable structure–shared resources or constrained resource allocation–and when certified bounds on policy quality matter, the regime where mathematical-programming and first-order solvers apply most directly.

In the weakly coupled MDP setting, we discuss two self-adapting approximations. The first has a construct stage that automatically formulates a linear program and the optimize stage relies on a commercial solver. It has no refine stage. The second constructs a different linear program, still optimizes using a MIP solver, but can refine the original linear program. In the general continuous state-action MDP setting, we also discuss two self-adapting approximations. Only the construct and optimize stages are engaged in the first: the construct stage builds a saddle point optimization model with a parameter that controls difficulty, and the optimize stage employs first-order methods that can exploit this parameter. The second extends the first approximation, and engages all three stages. It constructs a primal optimization problem with random features, a lightweight and extensible function approximator; then optimizes parameters using a commercial solver, and finally refines the model by adding more random features and constraints that guide the solution process with respect to the current approximation. In both settings, the user benefits from decades of solver development rather than building tailored solution approaches from scratch.

This tutorial builds on and complements several prior contributions in the INFORMS TutORials series. Powell and Topaloglu [26] covered approximate dynamic programming for large-scale resource allocation, while Ning [19] introduced affine MDPs and their tractable structure. Fu [13] connected MDPs to Monte Carlo tree search and AlphaGo. Mannor and Xu [15] addressed

data-driven methods for MDPs with parameter uncertainty, and Agrawal [3] surveyed advances in multiarmed bandits. Powell's tutorials on stochastic optimization [24, 25] provide a broader framing of sequential decision making under uncertainty. Sellmann [31] introduced meta-algorithms that automate algorithm configuration, an idea that resonates with our self-adapting theme. Most recently, Scully and Terenin [30] provided a design-principle view through the Gittins index. Our tutorial complements these by focusing on how optimization-based approximation methods can be made self-adapting, and how mature solvers can be leveraged at each stage of the COR cycle to reduce the burden on the user.

Finally, to complement the methodological discussion, the tutorial includes accompanying source code that serves as an executable companion to the main concepts. By providing a hands-on perspective, we help readers experience construction, optimization, and refinement as concrete procedures.

2. Background

This section introduces the modeling and optimization scaffolding that the remainder of the tutorial specializes. We first set up MDPs covering both finite-horizon and infinite-horizon discounted settings. We then present the linear-programming view of MDPs in finite-horizon form, since this is the form used in detail by §3, and indicate the infinite-horizon counterpart used in §4. We close with the COR cycle and a roadmap. Table 1 presents the notation used for later reference.

Table 1 Notation table.

Symbol	Meaning
$\mathcal{S}$, $\mathcal{A}$, $\mathcal{A}_s$	State space; action space; feasible actions in state s
$\mathcal{T}$, T	Decision periods; terminal period (finite horizon)
γ	Discount factor (infinite-horizon case)
$r_t(s,a)$, $\mathbb{P}_t(\cdot \mid s,a)$	One-period reward; transition distribution
π, π_t, $\pi_g(\cdot;V)$	Policy; period-t rule; greedy policy w.r.t. V
χ, $\mathrm{PV}_\chi(\pi)$	Initial state distribution; total expected reward
V_t, V_t^*, $V_t(s;\beta)$	Value function; optimal value function; its approximation
$\varphi_{t,n}$, N, β	Basis functions; their number; VFA coefficients
ν_t, $\rho_t(s,a)$	State-relevance distribution; occupation measure
$f_t(\beta;s,a)$, UB_ν, LB	Constraint-violation function; ALP upper/lower bounds
Weakly coupled MDPs (§3)	
$\mathcal{J}$, J	Component MDPs and their number
$\mathcal{K}$, K	Linking constraints and their number
$C_{t,k,j}(a_j)$, $b_{t,k}$	Resource use of action a_j; capacity of constraint k
$x_{t,j}(s_j,a_j)$, μ_t	Component marginal flow; coordination weights
$\mathcal{N}_t = (\mathcal{V}_t, \mathcal{E}_t)$	Feasibility network at period t
Continuous MDPs (§4)	
$y \in \mathcal{Y}$, $y^*_{\lambda,\beta}$	Sampling density; Gibbs density
λ, λ_t	KL-regularization weight and its schedule
(ϕ,ρ), θ_n	Random-feature family; sampled feature parameters
FALP_N, $\mathrm{FALP}_\mathrm{N}^\mathrm{SG}$	Feature-based ALP and self-guided ALP with N basis functions

2.1. Markov Decision Processes

An MDP is specified by a state space $\mathcal{S}$, an action space $\mathcal{A}$ with state-dependent feasible action sets $\mathcal{A}_s \subseteq \mathcal{A}$ for each $s \in \mathcal{S}$, a one-period reward function r_t mapping $\mathcal{S} \times \mathcal{A}$ to $\mathbb{R}$ at each period t, a transition function $\mathbb{P}_t(\cdot \mid s, a)$ giving the distribution of the next state given (s, a) at period t, and a horizon. The horizon may be finite, in which case decisions are made over periods $t \in \mathcal{T} := \{0, 1, \ldots, T-1\}$ with a terminal period T and no discounting. It may instead be infinite with a discount factor $\gamma \in (0, 1)$, in which case $r_t \equiv r$ and $\mathbb{P}_t \equiv \mathbb{P}$ are stationary.

We take each r_t to be deterministic only to lighten notation; this is without loss of generality for the risk-neutral objective below, since when the reward is random–written $r_t(s, a, \xi_t)$ for exogenous noise ξ_t–every formulation in this tutorial applies unchanged after replacing $r_t(s, a)$ by its conditional expectation $\bar{r}_t(s, a) := \mathbb{E}[r_t(s, a, \xi_t) \mid s, a]$ [27].

At each period, the decision maker observes the state, selects a feasible action, earns a reward, and transitions to a next state drawn from $\mathbb{P}_t(\cdot \mid s, a)$. A non-stationary policy assigns a map $\mathcal{S} \to \Delta(\mathcal{A})$ to each period, where $\Delta(\mathcal{A})$ denotes distributions over feasible actions; a stationary policy uses the same map at every period. Deterministic policies are the special case in which the map places unit mass on a single action. The total expected reward of a policy π given an initial state distribution χ is

$$\mathrm{PV}_\chi(\pi) := \mathbb{E}^\pi_\chi\left[\sum_{t \in \mathcal{T}} \gamma^t \, r_t(s^\pi_t, a^\pi_t)\right],$$

where s^π_t and a^π_t denote the random state and action at period t under policy π with $s^\pi_0 \sim \chi$, and the sum runs over $t \in \mathcal{T}$, with $T = \infty$ and $\mathcal{T} = \{0, 1, 2, \ldots\}$ in the infinite-horizon case, and $\gamma = 1$ in the finite-horizon case. The goal is to find a policy maximizing this quantity. Two sources of difficulty are the large or combinatorial state-action domains, and expensive expectation evaluations over large or continuous next-state distributions.

The supremum of $\mathrm{PV}_\chi(\pi)$ over policies is the expectation with repect to χ of the optimal value of a policy starting at period 0 and state s_0 and operating till the end of the horizon. An optimal policy value can be defined starting from any stage t and state s_t, that is, the best expected reward achievable starting from this stage and state. The value function $V_t(s_t)$ encodes these optimal policy values and satisfies the Bellman optimality equations [4]. Intuitively, the Bellman equations express the principle of optimality: the best one can do from a state is the immediate reward plus the best expected continuation value. In the finite-horizon case, the optimal value function is time-dependent, denoted V^*_t, and the Bellman equations translate to the backward recursion

$V_t^*(s) = \max_{a\in\mathcal{A}_s}\{r_t(s,a) + \mathbb{E}[V_{t+1}^*(s') \mid s,a]\}$ for $t \in \mathcal{T}$, with terminal condition $V_T^*(\cdot) := 0$. A non-stationary optimal policy $\pi_g(\cdot; V_t^*)$ at each period, where $\pi_g(s;V) \in \arg\max_{a\in\mathcal{A}_s}\{r_t(s,a) + \mathbb{E}[V(s') \mid s,a]\}$ denotes the policy acting greedily with respect to value function V. In the infinite-horizon discounted case, the optimal value function is stationary, denoted V^*, and the Bellman equation becomes the fixed-point $V^*(s) = \max_{a\in\mathcal{A}_s}\{r(s,a) + \gamma\,\mathbb{E}[V^*(s') \mid s,a]\}$, with a stationary optimal policy $\pi_g(\cdot; V^*)$ analogously defined by γ inserted.

2.2. Linear Programming Approach

We present the linear programming (LP) reformulation of the finite-horizon MDP. Specifically, the finite-horizon optimality equation is equivalent to a primal exact LP over the sequence of value functions $\{V_t\}_{t\in\mathcal{T}}$ per period:

$$\min_{\{V_t\}_{t\in\mathcal{T}}} \sum_{t\in\mathcal{T}} \mathbb{E}_{\nu_t}[V_t(s)] \;\text{ s.t. }\; V_t(s) - \mathbb{E}[V_{t+1}(s') \mid s,a] \geq r_t(s,a),\; \forall\,(t,s,a) \in \mathcal{T}\times\mathcal{S}\times\mathcal{A}_s, \quad \text{(P-ELP)}$$

where ν_t is a period-t state-relevance distribution defined over the state space and $V_T \equiv 0$. Here ν_t weights the states at which the value function should be more accurate. With full support, it leaves the exact optimum unchanged, but it becomes an important design choice once the value function is approximated. The reformulation works because the optimal value function is the *smallest* function satisfying all the (P-ELP) "Bellman" inequality constraints: any $\{V_t\}$ meeting the constraints is an upper bound on $\{V_t^*\}$, and minimizing the ν_t-weighted sum drives the values down until they coincide with V_t^* on the states ν_t weights.

By LP duality, the same problem has an equivalent dual that optimizes not over value functions but over the visitation frequencies a policy induces. The two views describe the same optimum but expose different structure, and the two settings later in the tutorial each build on one of them. The dual of (P-ELP) is a linear program in occupation-measure variables $\rho_t(s,a) \geq 0$ that, in the most common case where $\nu_0 = \chi$ and $\nu_t \equiv 0$ for $t \geq 1$, represent the expected frequency with which a policy visits state s and applies action a at period t under a policy starting from $s_0 \sim \chi$:

$$\begin{aligned}
\max_{\rho\geq 0} \quad & \sum_{t\in\mathcal{T}}\sum_{(s,a)} r_t(s,a)\,\rho_t(s,a) \\
\text{s.t.} \quad & \sum_a \rho_0(s,a) = \nu_0(s), && \forall\, s\in\mathcal{S}, \\
& \sum_a \rho_t(s,a) - \sum_{(s',a')} \mathbb{P}_{t-1}(s \mid s',a')\,\rho_{t-1}(s',a') = \nu_t(s), && \forall\, t\in\mathcal{T}\setminus\{0\},\; s\in\mathcal{S}.
\end{aligned} \quad \text{(D-ELP)}$$

The constraints are flow-balance equations: occupation mass entering state s at period t from any predecessor (s',a') at period $t-1$ matches mass leaving s across all actions, with the period-t

distribution ν_t acting as a source. An optimal policy is recovered from the optimal ρ^* by setting $\pi_t^*(a \mid s) = \rho_t^*(s,a)/\sum_{a'} \rho_t^*(s,a')$ at every (t,s) where the denominator is positive when ν_0 assigns a positive mass across the state space; states with zero denominator are unreachable under the optimal occupation measure starting from χ, and the policy can be assigned arbitrarily there.

Both (P-ELP) and (D-ELP) are exact, but their size scales with the state-action space, which is the difficulty that motivates approximation. We approximate the value function at each period by a linear combination of pre-specified *basis functions* $\varphi_{t,1}, \ldots, \varphi_{t,N}$ with coefficients $\beta_t = (\beta_{t,0}, \beta_{t,1}, \ldots, \beta_{t,N})$, giving $V_t(s;\beta_t) := \beta_{t,0} + \sum_{n=1}^{N} \beta_{t,n}\, \varphi_{t,n}(s)$ for $t \in \mathcal{T}$, with the convention $\beta_{T,0} := 0$ and $\varphi_{T,n} \equiv 0$ matching $V_T \equiv 0$. The basis functions may be nonlinear in s (for example, the monomials $1, s, s^2$ used in the inventory example of §4). Substituting into (P-ELP) replaces the function-valued decision variables with the finite coefficient vector $\beta := (\beta_t)_{t\in\mathcal{T}}$. It is convenient to isolate $\beta_{t,0}$ at each period via the *constraint violation function*

$$f_t(\beta; s, a) := \beta_{t+1,0} + r_t(s,a) - \sum_{n=1}^{N} \beta_{t,n}\, \varphi_{t,n}(s) + \sum_{n=1}^{N} \beta_{t+1,n}\, \mathbb{E}[\varphi_{t+1,n}(s') \mid s, a], \tag{1}$$

yielding the primal approximate linear program

$$\mathrm{UB}_\nu := \min_{\beta} \sum_{t\in\mathcal{T}} \mathbb{E}_{\nu_t}[V_t(s;\beta_t)] \quad \text{s.t.} \quad \beta_{t,0} \geq f_t(\beta; s, a), \quad \forall\, t \in \mathcal{T},\ (s,a) \in \mathcal{S} \times \mathcal{A}_s. \tag{P-ALP}$$

The constraint $\beta_{t,0} \geq f_t(\beta; s, a)$ asks the approximate value function to satisfy the Bellman inequality at (s,a); this is what the function f_t measures, and the pairs (s,a) at which the inequality fails are the violated constraints that the optimize stage must locate. When $\nu_0 = \chi$, the period-0 term $\mathbb{E}_\chi[V_0(s;\beta_0)]$ in the objective provides an upper bound on $\mathrm{PV}_\chi(\pi^*)$, so $\mathrm{PV}_\chi(\hat{\pi}) \leq \mathrm{PV}_\chi(\pi^*) \leq \mathbb{E}_\chi[V_0(s;\beta_0^*)]$ certifies the quality of any candidate policy $\hat{\pi}$ [11, 29].

The remainder of the tutorial works with two specializations. The weakly coupled setting of §3 works with the dual of (P-ALP), a relaxation of (D-ELP) in component-marginal flow variables. The continuous setting of §4 works with the stationary cost-form analogue of (P-ALP), where the period index drops, $\gamma \in (0,1)$ replaces unit discounting, and f has a $(1-\gamma)^{-1}$ scaling.

2.3. The COR Cycle

Obtaining a deployable policy from an MDP approximation requires an iterative process that we organize through the COR cycle. The **construct** stage instantiates the basis functions, the state-relevance distribution, and the optimization model itself, possibly through a reformulation that exposes useful structure. The **optimize** stage solves the resulting model. The approximate linear

programs (ALPs) above have too many constraints to enumerate; in particular, exponentially many in the number of components in the weakly coupled finite-horizon setting of §3, and infinitely many in the continuous setting of §4. Two classical routes tackle this challenge in both settings. *Constraint generation* solves a separation problem of the form $\max_{t,(s,a)} f_t(\hat{\beta}; s, a) - \beta_{t,0}$ at each iteration to identify a violated constraint (with the period index dropped in the infinite-horizon case), adds it to a restricted ALP, and re-solves. *Constraint sampling* draws a finite collection of state-action pairs from a user-chosen distribution and enforces the ALP constraints only at those points. The *refine* stage evaluates the quality of the current solution, either through a certified bound or through simulated policy performance, and feeds information back into the construct stage by enriching the basis, tightening the model, or adjusting the state-relevance distribution.

We reserve **refine** for adaptations that occur *across* solves of the optimization model, distinguishing them from in-solve adaptations that the optimize stage may itself perform and from one-time adaptations performed during construction. By this convention, a method that compresses its formulation once during construction and then solves it (the network relaxation in §3.2), or that adapts a sampling distribution during a single solve (constraint violation learning in §4.2), engages only construct and optimize, even though both methods adapt to instance information. Methods that engage refine, re-enter construct with an enriched model: a larger feasible-action support (delayed allocation in §3.3) or additional basis functions and guiding constraints (self-guided ALPs in §4.3).

Each stage imposes design decisions on the user. A narrow basis function class limits policy performance regardless of the optimization. Constraint generation requires a problem-specific separation routine that is combinatorially hard in the weakly coupled case and nonconvex in the continuous case; constraint sampling avoids separation but loses the lower-bound certificate and depends on a sampling distribution the user typically cannot calibrate in advance. The methods in this tutorial reduce user burden by automating decisions in one or more stages of the cycle. We refer to such methods as self-adapting approximations because the resulting procedure adjusts its construct, optimize, or refine stages to the structure of the problem instance, the data, or the solution progress.

The two settings that follow specialize this scaffold to complementary structures. The weakly coupled MDPs of §3 have a combinatorial state-action space that decomposes into small components linked by shared constraints; the structure is discrete, and the self-adapting methods there act on how feasibility is represented. The continuous MDPs of §4 have a state-action continuum with no such decomposition; the structure to exploit is the geometry of the constraint-violation function f, and the self-adapting methods there act on how the optimize stage searches for violated constraints. In both, the aim is to let the approximation adapt to the instance so the user hand-tunes less.

3. Weakly Coupled MDPs

Weakly coupled Markov decision processes (WMDPs) occupy an important middle ground between general MDPs, which often lack exploitable structure, and smaller or more structured systems that can be solved directly using standard dynamic programming. In a WMDP, rewards and state transitions decompose across components, while actions remain linked through shared feasibility constraints. This modeling paradigm arises whenever local decisions compete for limited global resources or must satisfy system-wide operational, safety, or regulatory requirements. Examples include assortment optimization, where products compete for limited display space [5, 9]; maintenance and replacement planning, where asset-level interventions must satisfy system-level operational restrictions [12]; and dynamic worker assignment, where jobs compete for shared workers [1].

The linking constraints create a fundamental computational challenge. Although each component MDP may be small, feasible decisions must be chosen jointly across all components, causing the feasible action space to grow combinatorially with the number of components. Approximation methods are therefore essential. A key question then becomes how to incorporate the linking constraints into the approximation: should feasibility be relaxed, enforced exactly, or represented only partially? After formalizing the problem in §3.1, we present two complementary approaches that address this question within the COR framework. The first, in §3.2, constructs a network whose paths correspond to feasible joint actions, allowing the feasible-action space to be represented compactly while preserving feasibility. The second, in §3.3, enforces feasibility through an explicitly maintained subset of feasible joint actions and expands this subset only when optimization indicates that additional actions are needed. Together, these methods illustrate two different ways of incorporating feasibility into tractable approximations.

3.1. Formulation

We study finite-horizon WMDPs consisting of J component MDPs indexed by $\mathcal{J} := \{1, \dots, J\}$. In each period $t \in \mathcal{T}$, the decision maker observes the state $s_j \in \mathcal{S}_{t,j}$ and selects an action $a_j \in \mathcal{A}_{t,j}$ of each component $j \in \mathcal{J}$, where $\mathcal{S}_{t,j}$ and $\mathcal{A}_{t,j}$ are assumed to be finite. The initial state s_j^0 at period $t = 0$ is fixed for all components j. The system's state and action at a period t are denoted by tuples $\mathbf{s} = (s_1, \dots, s_J) \in \prod_{j\in\mathcal{J}} \mathcal{S}_{t,j}$ and $\mathbf{a} = (a_1, \dots, a_J) \in \prod_{j\in\mathcal{J}} \mathcal{A}_{t,j}$, respectively. For each component $j \in \mathcal{J}$ and period $t \in \mathcal{T}$, selecting action $a_j \in \mathcal{A}_{t,j}$ in state $s_j \in \mathcal{S}_{t,j}$ yields reward $r_{t,j}(s_j, a_j)$ and transitions the component to state $s'_j \in \mathcal{S}_{t+1,j}$ with probability $\mathbb{P}_{t,j}(s'_j \mid s_j, a_j)$. In period t, the decision maker can only select a joint action $\mathbf{a} = (a_1, \dots, a_J)$ from the set

$$\mathcal{A}_t := \left\{ \mathbf{a} = (a_1, \dots, a_J) \in \prod_{j\in\mathcal{J}} \mathcal{A}_{t,j} \;\middle|\; \sum_{j\in\mathcal{J}} C_{t,k,j}(a_j) \le b_{t,k}, \quad \forall k \in \mathcal{K} \right\}.$$

That is, the joint action is feasible only if the component actions satisfy the linking constraints. Each constraint $k \in \mathcal{K}$ represents a shared system-wide requirement, such as a budget or capacity. Under action a_j, component j contributes $C_{t,k,j}(a_j)$ units to constraint k. The total contribution across all components must therefore satisfy $\sum_{j\in\mathcal{J}} C_{t,k,j}(a_j) \le b_{t,k}$, where $b_{t,k}$ is the available amount of resource k in period t. The set $\mathcal{K} := \{1,\dots,K\}$ indexes these linking constraints.

Given the state space $\mathcal{S}_t := \prod_{j\in\mathcal{J}} \mathcal{S}_{t,j}$ and the feasible action set $\mathcal{A}_t$ at period t, the objective is to find a policy $\boldsymbol{\pi} := (\pi_t)_{t\in\mathcal{T}}$ that selects a feasible joint action after observing the current states of all components. Specifically, each decision rule π_t maps a joint state $\mathbf{s} \in \mathcal{S}_t$ to an action $\pi_t(\mathbf{s}) \in \mathcal{A}_t$. An optimal policy maximizes the total expected reward $\mathrm{PV}_\chi(\boldsymbol{\pi})$ over the horizon $\mathcal{T}$, starting from an initial state distribution χ. In our setting, χ is concentrated on the initial state $\mathbf{s}_0$. The policy can be obtained by solving the optimality equations

$$V_t(\mathbf{s}) = \max_{\mathbf{a}\in\mathcal{A}_t} \left\{ \sum_{j\in\mathcal{J}} r_{t,j}(s_j,a_j) + \sum_{\mathbf{s}'\in\mathcal{S}_{t+1}} \mathbb{P}_t(\mathbf{s}' \mid \mathbf{s},\mathbf{a})\, V_{t+1}(\mathbf{s}') \right\}, \quad \forall t \in \mathcal{T},\ \mathbf{s}\in\mathcal{S}_t, \tag{WMDP}$$

with terminal condition $V_T(\cdot) = 0$ and weakly coupled transitions $\mathbb{P}_t(\mathbf{s}'|\mathbf{s},\mathbf{a}) = \prod_{j\in\mathcal{J}} \mathbb{P}_{t,j}(s'_j|s_j,a_j)$.

EXAMPLE 1 (DYNAMIC ASSORTMENT OPTIMIZATION [9]). Dynamic assortment optimization is a classical revenue management problem in which a retailer repeatedly chooses which subset of products to offer while learning about demand. In a WMDP formulation, each component MDP corresponds to one product, and the action $a_j \in \{0,1\}$ indicates whether product j is offered in period t. For example, with three products, feasible assortments may be given by

$$\mathcal{A}_t = \left\{ \mathbf{a} \in \{0,1\}^3 : 2a_1 + 10a_2 + 8a_3 \le 14,\ \ 3a_1 + 18a_2 + 15a_3 \le 20 \right\}, \tag{2}$$

where the first and second constraints represent two shared resources, such as display space and available inventory, with capacities 14 and 20, respectively. Offering products 1, 2, and 3 consumes $(2, 10, 8)$ units of display space and $(3, 18, 15)$ units of inventory, respectively. As a result, some combinations of products are feasible while others violate one or both resource limits. □

3.2. Self-Adapting Network Relaxations (Construct + Optimize)

We begin with a self-adapting construction that addresses the challenge of representing the large feasible-action space. Rather than working directly with the linking constraints, the method builds a network that encodes all feasible joint actions. Feasibility is then enforced automatically: every source-to-sink path in the network corresponds to a feasible joint action, and every feasible joint action corresponds to a path. This allows the approximation to reason about feasible decisions

through the network rather than through an explicit representation of the combinatorial action space. Moreover, the size of the network adapts to the structure of the linking constraints and can be much smaller than the feasible-action space itself. The resulting network yields an LP representation, so once the network is constructed, the **optimize** step of COR reduces to solving an LP/MIP.

To see the **construct** step in detail, consider the specialization of the dual LP formulation (D-ELP) for a general MDP to the WMDP in §3.1. A direct formulation requires occupation measures $\rho_t(\mathbf{s}, \mathbf{a})$ defined over the joint state-action space, whose size grows rapidly with the number of components. A natural way to exploit the decomposable structure of a WMDP is therefore to replace these joint variables with component-level marginal flows. Specifically, let $x_{t,j}(s_j, a_j)$ denote the probability that component j is in state s_j and takes action a_j in period t. This substitution dramatically reduces the size of the LP because the variables now scale with the component state and action spaces rather than the full joint space. The trade-off is that the formulation no longer tracks how component actions are combined into a joint action. Consequently, the linking constraints cannot be enforced for every realized state-action pair and instead hold only in expectation. The resulting model is the marginalized-flow relaxation of Adelman and Mersereau [2], specialized to the weakly coupled structure of (D-ELP). It is also equivalent to the LP dual of a Lagrangian relaxation of (WMDP) obtained by dualizing the linking constraints.

Formally, define $\mathcal{I}_t := \{(j, s_j, a_j) : j \in \mathcal{J},\ s_j \in \mathcal{S}_{t,j},\ a_j \in \mathcal{A}_{t,j}\}$ for all $t \in \mathcal{T}$, and let $\mathbf{x} := (x_{t,j})_{t\in\mathcal{T},, j\in\mathcal{J}}$ denote the collection of component marginal flows. Substituting the joint occupation measures in (D-ELP) with these marginal variables yields the following formulation:

$$\max_{\mathbf{x}\geq 0} \quad \sum_{t\in\mathcal{T}} \sum_{(j,s_j,a_j)\in\mathcal{I}_t} r_{t,j}(s_j, a_j)\, x_{t,j}(s_j, a_j) \tag{LAG}$$

$$\text{s.t.} \quad \sum_{a_j\in\mathcal{A}_{0,j}} x_{0,j}(s_j^0, a_j) = 1, \qquad \forall j \in \mathcal{J}, \tag{3a}$$

$$\sum_{a_j\in\mathcal{A}_{t,j}} x_{t,j}(s_j, a_j) = \sum_{s'_j\in\mathcal{S}_{t-1,j}} \sum_{a'_j\in\mathcal{A}_{t-1,j}} \mathbb{P}_{t-1,j}(s_j \mid s'_j, a'_j)\, x_{t-1,j}(s'_j, a'_j),$$

$$\forall t \in \mathcal{T}\setminus\{0\}, j \in \mathcal{J}, s_j \in \mathcal{S}_{t,j}, \tag{3b}$$

$$\sum_{(j,s_j,a_j)\in\mathcal{I}_t} C_{t,k,j}(a_j)\, x_{t,j}(s_j, a_j) \leq b_{t,k}, \qquad \forall t \in \mathcal{T}, k \in \mathcal{K}. \tag{3c}$$

The objective maximizes expected reward across all periods, as in (D-ELP). Constraints (3a)-(3b) are the component-level analogues of the flow-balance constraints in (D-ELP): (3a) initializes each component at its starting state, while (3b) propagates marginal flow according to the component

transition probabilities. Constraint (3c) couples the components by enforcing the linking constraints on the marginal flows. Because the variables represent marginal probabilities rather than joint occupation measures, the resource consumption in (3c) is averaged over the component distributions; thus, the linking constraints are enforced only in expectation. This relaxation is tractable because its size depends on the component state and action spaces rather than the full joint state-action space.

The marginalized-flow relaxation (LAG) has become a foundational and tractable approximation for WMDPs [7, 8, 36]. Its optimal objective upper bounds the WMDP optimal policy value. The tightness of this bound depends on how well the marginal flows reflect the underlying feasible-action set. Specifically, because (3c) enforces the linking constraints only in expectation, the solution may assign probability mass to combinations of component actions that cannot be implemented simultaneously. As a result, converting the relaxation into feasible decisions often requires additional steps such as sampling, repair, rounding, or reoptimization. This issue is pronounced when the feasible-action set has rich combinatorial structure [1, 18]. The optimal solution to (LAG) with such repair defines a "fluid" policy as illustrated in Example 2, which builds on Example 1.

EXAMPLE 2. Recall the action set $\mathcal{A}_t$ in (2):

$$\mathcal{A}_t = \left\{\mathbf{a} \in \{0,1\}^3 : 2a_1 + 10a_2 + 8a_3 \le 14,\ \ 3a_1 + 18a_2 + 15a_3 \le 20\right\}.$$

The set of feasible actions is $\{(0,0,0),(1,0,0),(0,1,0),(0,0,1),(1,0,1)\}$. Fix a state $\mathbf{s} = (s_1, s_2, s_3)$ and consider the following component marginals,

$$\begin{aligned} &x_1(s_1,0) = 0, && x_2(s_2,0) = \frac{1}{2}, && x_3(s_3,0) = \frac{1}{2},\\ &x_1(s_1,1) = 1, && x_2(s_2,1) = \frac{1}{2}, && x_3(s_3,1) = \frac{1}{2}. \end{aligned}$$

The marginals satisfy the linking constraints in expectation because $2x_1(s_1,1) + 10x_2(s_2,1) + 8x_3(s_3,1) = 2(1) + 10\left(\frac{1}{2}\right) + 8\left(\frac{1}{2}\right) = 11 \le 14$, and $3x_1(s_1,1) + 18x_2(s_2,1) + 15x_3(s_3,1) = 3(1) + 18\left(\frac{1}{2}\right) + 15\left(\frac{1}{2}\right) = 19.5 \le 20$. As discussed in §2.2, a standard policy using such measures is

$$\pi_j(a_j \mid s_j) = \frac{x_j(s_j,a_j)}{\sum_{a'_j \in \mathcal{A}_{t,j}} x_j(s_j,a'_j)} = x_j(s_j,a_j).$$

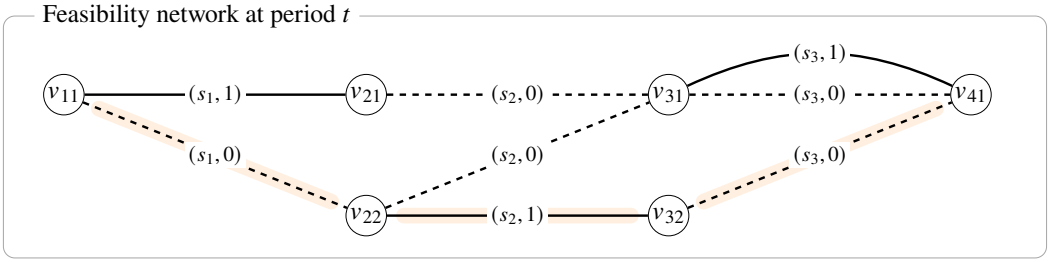
We have $\pi_1(1 \mid s_1) = 1$, $\pi_2(1 \mid s_2) = 1/2$, and $\pi_3(1 \mid s_3) = 1/2$. However, independently sampling each component action from these policies does not always produce a feasible joint action. Indeed, every feasible action in $\mathcal{A}_t$ satisfies $a_1 + a_2 \le 1$, but independent sampling produces $a_1 = a_2 = 1$ with probability $\pi_1(1 \mid s_1)\pi_2(1 \mid s_2) = 1 \cdot (1/2) = 1/2$. Thus, although the component occupation

measures satisfy the expected resource constraints, their induced policy is not supported on the feasible joint action set. Therefore, additional repair or projection is needed to recover feasibility. □

Feasibility network relaxations [FNR; 17, §4] aim to address the fundamental feasibility issue in Example 2 by preserving the tractability of marginal flows while incorporating stronger information about the combinatorial structure of the linking constraints. For each period t, the relaxation builds a feasibility network whose source-to-sink paths have a one-to-one correspondence with actions in $\mathcal{A}_t$.

Formally, let $\mathcal{N}_t = (\mathcal{V}_t, \mathcal{E}_t)$ denote an acyclic layered network with node set $\mathcal{V}_t$, arc set $\mathcal{E}_t$, and a designated source node and sink node. Each layer corresponds to a component $j \in \mathcal{J}$, arcs correspond to component state-action pairs $(s_j, a_j) \in \mathcal{S}_{t,j} \times \mathcal{A}_{t,j}$, and each source-to-sink path encodes a state $\mathbf{s} \in \mathcal{S}_t$ and feasible joint action $\mathbf{a} \in \mathcal{A}_t$. Traversing the network sequentially builds a joint state-action pair across components while enforcing the linking constraints. The next example illustrates this approach using the running Example 1.

Figure 1 **Network for Example 3. Component state-action pairs (s_j, a_j) are depicted as labels on arcs in layer j. Nodes $v_{\ell,k}$ are indexed by layer $\ell \in \{1, \ldots, J+1\}$ and position k within the layer, with $\ell = 1$ the source layer and $\ell = J+1$ the sink layer. Solid and dashed arcs denote $a_j = 0$ and $a_j = 1$, respectively.**



EXAMPLE 3. Consider once again the dynamic assortment setting of Example 1 and its action set $\mathcal{A}_t = \{(0,0,0), (1,0,0), (0,1,0), (0,0,1), (1,0,1)\}$. At a fixed period t, suppose for simplicity that each component j has a single state $s_j \in \mathcal{S}_{t,j}$. The network in Figure 1 represents $\mathcal{A}_t$: an arc in layer j corresponds to a choice of action a_j, and there is a one-to-one correspondence between source-to-sink paths from v_{11} to v_{41} and feasible joint actions in $\mathcal{A}_t$. In particular, the highlighted bottom path corresponds to the component state-action pairs $(s_1, 0)$, $(s_2, 1)$, and $(s_3, 0)$, and therefore to the joint state $\mathbf{s} = (s_1, s_2, s_3)$ and feasible action $(0,1,0) \in \mathcal{A}_t$. □

Given a linear description of the feasible action set, such as (2), the associated network can be generated automatically. Specifically, [17, §3.1–3.2] build the network layer by layer as follows. Starting from a root node v_{11}, the procedure considers the components $j = 1, 2, \ldots, J$ one at a time.

For each component j, the algorithm extends each existing path by adding an arc associated with each action $a_j \in \mathcal{A}_{t,j}$ that points to a new node. Each new node records the remaining capacity in the linking constraints, accounting for the actions selected so far.

For example, consider the action $a_1 = 1$ on the arc ending at node v_{21} in Figure 1. This action consumes 2 and 3 units of the first and second linking constraints, respectively. Since the initial capacities are 14 and 20, the remaining capacities at v_{21} are $14-2=12$ and $20-3=17$. If an expansion leads to negative remaining capacity in any linking constraint, then the corresponding partial action vector is infeasible, and the node can be discarded. Otherwise, nodes in the same layer with identical remaining capacities can be merged, because the feasible set of actions starting from those nodes is the same, as they only depend on the remaining constraint capacities. This merging step is what makes the network construction self-adapting: the resulting network's size depends on the structure of the linking constraints in the specific instance rather than on a fixed enumeration of joint actions. Additional properties that can further reduce the size of the network are explored in [17].

Once a network is generated, the FNR formulation augments the marginalized-flow model with network flow variables. The key idea is to require the marginal flows $\mathbf{x}$ in (LAG) to be consistent with feasible paths in the network. In other words, the probability mass assigned to component state-action pairs must be routable through arcs that belong to feasible source-to-sink paths. Let $\mathcal{E}_t$ be the set of arcs in the FNR network at time t. For each period $t \in \mathcal{T}$ and arc $e \in \mathcal{E}_t$, let $y_{t,e} \geq 0$ denote the flow on arc e. For each component $j \in \mathcal{J}$ and state-action pair $(s_j, a_j) \in \mathcal{S}_{t,j} \times \mathcal{A}_{t,j}$, let $\mathcal{E}_t(j, s_j, a_j) \subseteq \mathcal{E}_t$ denote the set of arcs in layer j labeled by (s_j, a_j). Finally, let $\delta_t^+(v)$ and $\delta_t^-(v)$ denote the outgoing and incoming arcs of node v, respectively. The FNR formulation keeps the component flow-balance constraints (3a)-(3b) from (LAG) and replaces the in-expectation linking constraint (3c) with the following network-consistency constraints:

$$\sum_{e \in \mathcal{E}_t(j,s_j,a_j)} y_{t,e} = x_{t,j}(s_j, a_j), \qquad t \in \mathcal{T},\ j \in \mathcal{J},\ s_j \in \mathcal{S}_{t,j},\ a_j \in \mathcal{A}_{t,j}, \tag{4a}$$

$$\sum_{e \in \delta_t^+(v)} y_{t,e} = \sum_{e \in \delta_t^-(v)} y_{t,e}, \qquad t \in \mathcal{T},\ v \in \mathcal{V}_t \setminus \{\text{source, sink}\}. \tag{4b}$$

Constraint (4a) ties the two flow systems together by requiring that the marginal probability assigned to component action (s_j, a_j) exactly match the amount of network flow passing through arcs labeled by that same state-action pair. Constraint (4b) is flow conservation at every internal node, which forces the network flow $\mathbf{y}_t := (y_{t,e})_{e \in \mathcal{E}_t}$ to decompose into a sum of source-to-sink path flows. Since each source-to-sink path encodes a feasible joint action by construction, $\mathbf{y}_t$ is implicitly

a distribution over feasible joint actions, and (4a) ties the marginals $\mathbf{x}_t$ to that distribution. Since the marginal mass on (s_j, a_j) is constrained to come from a path flow over feasible joint actions, FNR rules out the infeasibility issue in (LAG) noted earlier. It is shown in [17] that FNR achieves the same lower-bound quality as the approximate linear programming (ALP) formulation of Schweitzer and Seidmann [29] and de Farias and Van Roy [11], but typically admits a more structured and much smaller LP representation. A similar case holds with respect to the alternative Lagrangian approach proposed in [6]. This is because the network adapts to the linking constraints, and can be shown to be polynomial-sized or computationally tractable in several structured cases [17, §5.2].

Policy. The network representation also suggests a simple way to extract a feasible policy. At each period, once the FNR linear program is solved, one can select any source-to-sink node path with positive flow and implement the corresponding joint action. Because every such path encodes a feasible joint state-action pair, this procedure produces actions that satisfy the linking constraints. When the LP solution distributes flow across multiple paths, missing components can be "repaired" by computing a longest path on the network using the optimal duals of the linear program. It can be shown that this policy is equivalent to applying a *greedy action*, that is, choosing a feasible joint action that maximizes the current reward plus the continuation value implied by the approximate value function returned by the linear program.

Summary. In the context of COR, the *construct* step builds a feasibility network that exploits the combinatorial structure of the linking constraints to reduce representation size. The *optimize* step, in turn, solves the resulting linear program to obtain a *guaranteed* valid lower bound on the optimal policy cost, as well as a policy from the network flows.

Limitations. The main limitation of FNR is that the size of the reformulation is determined by the combinatorial structure of the feasible-action set. The feasibility network can be substantially smaller than an explicit enumeration of feasible joint actions, especially when the linking constraints create opportunities for compression, as depicted in the examples above. Nadarajah and Cire [17] also identify settings in which the minimal network is polynomially sized in the number of components, and the resulting linear program is tractable. However, this compression is instance-dependent. In problems with many components, many linking constraints, or limited symmetry in feasible action sets, the network may still become large, making its construction and subsequent LP solution computationally demanding. In such cases, extensions of the Lagrangian fluid policy to group structure [10] and to the average cost setting [35] are worth exploring.

3.3. Delayed Allocation and Vertex Policies (Construct + Optimize + Refine)

FNR removes the infeasibility of (LAG) by explicitly representing the feasible-action set through a network. The price of this stronger representation is that the network can become large when the feasible-action set is complex. Delayed allocation [18, §4] pursues the same goal–tying the marginal flows to a distribution over feasible joint actions–without constructing a complete representation of the feasible-action set. Instead, it starts with a small restricted set of feasible joint actions and expands this set only when optimization indicates that additional actions would improve the model. In the language of COR, this introduces an explicit **refine** step: rather than building a large representation upfront, the model is strengthened progressively by adding feasible actions as needed.

Construct. The difficulty that (LAG) runs into, illustrated in Example 2, is that the marginal flows $x_{t,j}(s_j, a_j)$ record how often each component takes each action but say nothing about which component actions are taken *together*. Two component actions can each be acceptable on their own yet be impossible to combine. To rule this out, we add bookkeeping that tracks joint actions explicitly. For each period t, let $\hat{\mathcal{A}}_t \subseteq \mathcal{A}_t$ be a restricted set of feasible joint actions, and introduce a variable $\mu_t(\mathbf{a}) \geq 0$ for each $\mathbf{a} \in \hat{\mathcal{A}}_t$, interpreted as the probability mass assigned to feasible joint action $\mathbf{a}$ in period t. Because every action in $\hat{\mathcal{A}}_t$ is feasible by construction, any distribution over $\hat{\mathcal{A}}_t$ is automatically a distribution over feasible joint actions–precisely the information the marginals alone cannot carry.

The delayed-allocation model keeps the objective and the per-component flow-balance constraints (3a)–(3b) of (LAG), and replaces the in-expectation linking constraint (3c) with

$$\sum_{s_j \in \mathcal{S}_{t,j}} x_{t,j}(s_j, a_j) \leq \sum_{\mathbf{a}' \in \hat{\mathcal{A}}_t : a'_j = a_j} \mu_t(\mathbf{a}'), \qquad \forall t \in \mathcal{T},\ j \in \mathcal{J},\ a_j \in \hat{\mathcal{A}}_{t,j}, \tag{5a}$$

$$\sum_{\mathbf{a}' \in \hat{\mathcal{A}}_t} \mu_t(\mathbf{a}') = 1, \qquad \forall t \in \mathcal{T}. \tag{5b}$$

Constraint (5b) makes μ_t a probability distribution over the represented feasible actions in $\hat{\mathcal{A}}_t$. The purpose of constraint (5a) is to ensure that the component-level marginals are consistent with this distribution. Intuitively, if the marginal flows indicate that component j takes action a_j with some probability, then the represented feasible actions must contain enough probability mass on joint actions whose jth component is also a_j. Otherwise, the marginal flows would be describing behavior that cannot be assembled from the represented feasible actions.

The effect of constraint (5a) is easiest to see through a simple example.

EXAMPLE 4. Recall the assortment setting of Example 1, with $\mathcal{A}_t = \{(0,0,0), (1,0,0), (0,1,0), (0,0,1), (1,0,1)\}$, and suppose that the current represented set is

$\hat{\mathcal{A}}_t = \{(0,0,0),(0,1,0)\}$, with weights $\mu_t(0,0,0) = 0.7$ and $\mu_t(0,1,0) = 0.3$. Since only the action $(0,1,0)$ offers product 2, constraint (5a) implies that $\sum_{s_2} x_{t,2}(s_2,1) \le 0.3$. Similarly, because neither represented action offers product 1 or product 3, we have $\sum_{s_1} x_{t,1}(s_1,1) \le 0$, and $\sum_{s_3} x_{t,3}(s_3,1) \le 0$.

Thus, the marginal flows can place probability mass only on component actions that are supported by the represented feasible actions. The benefit is that feasibility comes automatically: sampling a joint action from μ_t can only produce $(0,0,0)$ or $(0,1,0)$, both of which are feasible. This is precisely what was missing in Example 2, where marginal flows that satisfied the linking constraints only in expectation could still generate the infeasible action $(1,1,0)$ through independent sampling. □

Optimize. Solving the delayed-allocation model returns the marginal flows $x_{t,j}$ and the action weights μ_t. This model is a restricted version of the case $\hat{\mathcal{A}}_t = \mathcal{A}_t$, which attains the FNR bound. Starting from a subset of feasible actions, the delayed-allocation objective increases toward the FNR bound as actions are added. The gap between these bounds signals that the represented set is still missing useful actions–exactly what the refine step diagnoses.

Refine. We can now make precise which action to add. After solving the delayed allocation model, read off the dual variables of the two coupling constraints: let $d^*_{t,j}(a_j)$ be the dual of (5a) and ξ the dual of (5b). The first measures the marginal value of relaxing the represented set's support for component action a_j–how much the objective would rise if a little more feasible-action mass containing a_j were available–while ξ prices the unit probability budget in (5b). A feasible joint action $\mathbf{a}$ not yet in $\hat{\mathcal{A}}_t$ would, if added, let the model place weight on $\mathbf{a}$; the resulting change in objective per unit of weight is its *reduced cost*, $\sum_j d^*_{t,j}(a_j) - \xi$, the combined value of the component actions it makes available net of the budget price. Adding $\mathbf{a}$ helps exactly when this quantity is positive. Searching for the best action to add is therefore the pricing problem

$$R_t := \max_{\mathbf{a} \in \mathcal{A}_t} \sum_{j \in \mathcal{J}} d^*_{t,j}(a_j) - \xi, \tag{6}$$

a deterministic optimization over the original feasible-action set $\mathcal{A}_t$. If $R_t > 0$, the maximizing action is added to $\hat{\mathcal{A}}_t$ and the model is re-solved; if $R_t \le 0$, no feasible action improves the current model and refinement stops. Readers familiar with linear programming will recognize this loop as column generation, with each represented action a column and (6) the pricing step. Repeating it over periods yields the refinement that enlarges the represented set only where it strengthens the model.

This is the sense in which allocation is *delayed*. The Lagrangian relaxation (LAG) allocates probability mass over the relaxed marginals first and restores feasibility afterward through repair or

projection. Delayed allocation reverses the order: it identifies feasible joint actions through pricing first, and only then allocates mass over their convex hull. Probability is committed to combinations of actions only after those combinations are known to be feasible.

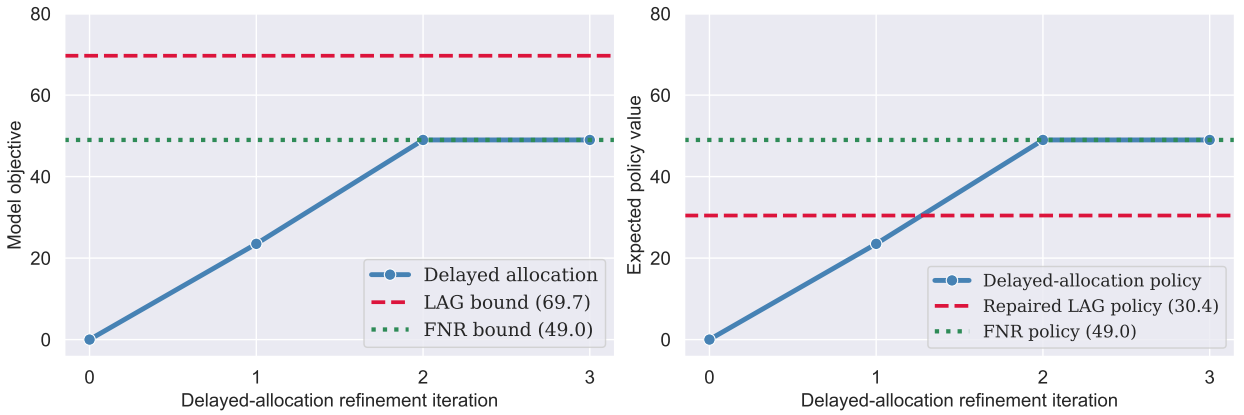
Policy. The weights μ_t also deliver a feasible policy directly. Because they place mass only on feasible joint actions, the policy can sample a joint action from μ_t rather than sampling each component independently from its marginal, as the fluid policy from (LAG) does. Nadarajah and Cire [18, Definition 3] call this a *vertex policy* and show it is feasible at every state by construction, in contrast to the fluid policy, which can produce infeasible joint actions and require ad hoc repair. The vertex policy admits a randomized form, sampling from μ_t^*, and a deterministic form, playing the action with the largest $\mu_t^*(\mathbf{a})$ in its support.

EXAMPLE 5. The feasible joint actions, corresponding to the v_{11}–v_{41} paths in Figure 1, are $\mathcal{A}_0 = \{(0,0,0),(1,0,0),(0,1,0),(0,0,1),(1,0,1)\}$, the same in every period. Initially all products are in state s_j^0. In period 0 the rewards from offering products 1, 2, and 3 are $r_{0,1}(s_1^0,1) = 1$, $r_{0,2}(s_2^0,1) = 22$, and $r_{0,3}(s_3^0,1) = 0.5$. If product j is offered in period 0, it transitions to state $\hat{s}_j$ in period 1 with probabilities $\mathbb{P}(\hat{s}_1 \mid s_1^0,1) = 0.5$, $\mathbb{P}(\hat{s}_2 \mid s_2^0,1) = 0.5$, and $\mathbb{P}(\hat{s}_3 \mid s_3^0,1) = 1$, with period-1 rewards $r_{1,1}(\hat{s}_1,1) = 42$, $r_{1,2}(\hat{s}_2,1) = 32$, and $r_{1,3}(\hat{s}_3,1) = 11$; products not offered remain in s_j^0.

We initialize delayed allocation with only the reference "offer-nothing" action $\mathbf{a}^{\mathrm{F}} = (0,0,0)$, that is, $\hat{\mathcal{A}}_t = \{\mathbf{a}^{\mathrm{F}}\}$ for all $t \in \mathcal{T}$. The restricted model forces every component to choose action zero, yielding objective value 0 and marginal flows $x_{t,j}^*(s_j^0,0) = 1$ for all $t \in \mathcal{T}$ and $j \in \mathcal{J}$. The optimal duals of (5a) at $t = 0$ are $d_{0,j}^*(0) = 0$ for all j, $d_{0,1}^*(1) = 1$, $d_{0,2}^*(1) = 22$, $d_{0,3}^*(1) = 0.5$, and $\xi = 0$, so only component actions with $a_j = 1$ carry positive value. The pricing problem (6) in period 0 selects $\mathbf{a}^+ = (0,1,0)$ with value 22, which is added to $\hat{\mathcal{A}}_0$; the same reasoning identifies $(1,0,1)$ for period 1. Solving the refined model gives objective value 23.5.

Figure 2 traces the refinement in both bounds (left) and policy values (right), alongside (LAG) and FNR. Starting from $(0,0,0)$, the initial bound and policy value are both zero. After the second refinement, delayed allocation reaches the FNR value of 49, matching both the FNR bound and the FNR policy value; the third refinement adds no improving action, so the represented set is already sufficient. The Lagrangian bound stays higher because it relaxes feasibility in expectation, and its repaired policy stays below the self-adapting policies. For reference, the model sizes (variables plus constraints) of (LAG), FNR, and the converged delayed-allocation LP are 51, 127, and 73; the FNR bound and policy value are recovered with 57% of FNR's model size. □

Figure 2 **Progression of delayed-allocation bounds (left) and policy values (right).**



The pricing step can itself be approximate. When (6) is expensive to solve exactly, an α-approximation oracle may be used in its place; Nadarajah and Cire [18, Theorem 1] show that the resulting bound remains within a factor $1/\alpha$ of the strongest formulation in this class. The oracle can be an approximation algorithm, a greedy or local-search procedure, or a rounding heuristic over the linking constraints, keeping refinement practical even when exact pricing is hard.

Summary. Delayed allocation follows the COR cycle directly. *Construct* a restricted LP over a small set of feasible joint actions, with μ_t supplying the coordination the marginals lack. *Optimize* it to obtain marginal flows and action weights. *Refine* the represented set by adding feasible actions that can improve the current model. Compared with FNR, adaptation occurs later: FNR compresses the feasibility structure during construction, whereas delayed allocation learns during refinement which feasible actions are worth representing explicitly.

Limitations. The effectiveness of delayed allocation depends on the ability to efficiently identify promising feasible actions that are not yet represented. This search is carried out through the problem (6). When it is difficult, refinement can become the computational bottleneck, and the method may terminate with a compact but weak restricted formulation. Moreover, as discussed earlier, the delayed allocation linear program does not return an upper bound on the optimal policy value, unless exact pricing is possible.

Getting started with code. Code accompanying this section is provided in the tutorial repository under the folder `weakly-coupled`[1]. This folder contains three notebooks, organized in the order of Sections 3.2 and 3.3, that illustrate the methods on a dynamic assortment instance.

[1] Code is included with the review package. A public GitHub repository will be made available after the review process.

4. Continuous Markov Decision Processes

Continuous MDPs arise in supply chain management, healthcare, energy, and finance, where state and action spaces are naturally continuous and discretization either introduces approximation error or requires prohibitively large grids. The COR cycle for such MDPs faces challenges absent from the weakly coupled setting of §3. Without an exploitable coupling structure, the basis functions in the stationary VFA of §2.2 must be designed from scratch, and optimization algorithms must directly manage the continuum of state-action pairs without relying on naive discretization. The two classical solution routes for the ALP discussed in §2.2 both struggle in this setting: constraint generation requires solving a global nonconvex separation problem at every iteration, while constraint sampling requires a sampling distribution whose effective form depends on the unknown violation landscape.

This section presents two self-adapting methods for different stages of the COR cycle. *Constraint violation learning* (§4.2) reformulates the ALP as a regularized saddle-point problem and uses a primal-dual first-order algorithm to learn the violation landscape during optimization, replacing the user-chosen constraint-sampling distribution with an endogenous one. *Self-guided approximate linear programs* (§4.3) automate basis-function design by sampling random features and refining the approximation across iterations using guiding constraints.

4.1. Approximate Linear Programming

We specialize the generic MDP setup of §2.1 to the infinite-horizon discounted continuous setting and switch from reward maximization to cost minimization, following the convention of the underlying literature [14, 22]. The state space is $\mathcal{S} \subseteq \mathbb{R}^{d_\mathcal{S}}$ and the action space is $\mathcal{A} \subseteq \mathbb{R}^{d_\mathcal{A}}$ with feasible action sets $\mathcal{A}_s \subseteq \mathcal{A}$, the one-period cost is $c = -r$, and $\gamma \in (0,1)$ is the discount factor. The expected discounted cost of a policy π from initial distribution χ is $\mathrm{PV}_\chi(\pi) := \mathbb{E}^\pi_\chi\left[\sum_{t=0}^\infty \gamma^t c(s^\pi_t, a^\pi_t)\right]$. Negating reward and flipping the sense in (P-ALP) of §2.2, and specializing to the stationary infinite-horizon form (period index dropped, $\gamma \in (0,1)$ in the optimality equation, and a $(1-\gamma)^{-1}$ scaling in the constraint violation function), yields the cost-form optimamlity equation and primal ALP

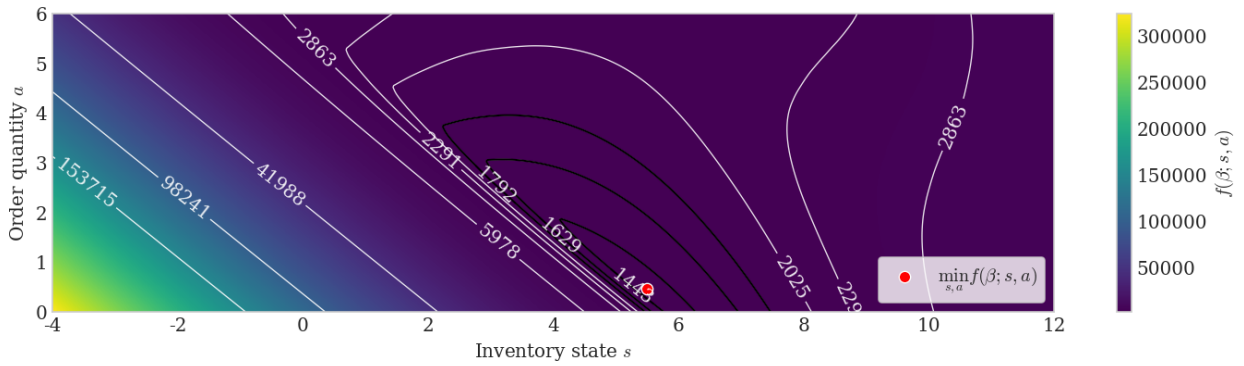
$$V^*(s) = \min_{a \in \mathcal{A}_s} \{c(s,a) + \gamma\, \mathbb{E}[V^*(s') \mid s, a]\}, \qquad s \in \mathcal{S}, \tag{7}$$

$$\mathrm{LB}_\nu := \max_{\beta \in \mathbb{R}^{N+1}} \left\{\mathbb{E}_\nu[V(s;\beta)] \text{ s.t. } \beta_0 \le f(\beta; s, a),\ \forall (s,a) \in \mathcal{S} \times \mathcal{A}_s\right\}, \tag{ALP}$$

where the stationary VFA $V(s;\beta) = \beta_0 + \sum_n \beta_n \varphi_n(s)$ is unchanged over time, ν is called a state-relevance distribution defined over $\mathcal{S}$, and the constraint violation function is

$$f(\beta; s, a) := \frac{1}{1-\gamma}\left(c(s,a) - \sum_{n=1}^N \beta_n(\varphi_n(s) - \gamma\, \mathbb{E}[\varphi_n(s') \mid s, a])\right). \tag{8}$$

Figure 3 **Contour plot of the constraint violation function $f(\beta;\cdot,\cdot)$ for Example 6 at a representative β.**



Thus, in continuous MDPs, ALP is a semi-infinite linear program: it has $N+1$ decision variables and a continuum of constraints, one for each state-action pair. As mentioned in §2.2, when $\nu = \chi$, LB_χ provides a lower bound on $\mathrm{PV}_\chi(\pi^*)$, so if combined with the upper bound $\mathrm{PV}_\chi(\hat{\pi})$, we obtain $\mathrm{LB}_\chi \le \mathrm{PV}_\chi(\pi^*) \le \mathrm{PV}_\chi(\hat{\pi})$ for any candidate policy $\hat{\pi}$. The inventory Example 6 illustrates the constraint violation function and serves as the running example for both methods in §4.2–§4.3.

EXAMPLE 6 (INVENTORY CONTROL UNDER STOCHASTIC DEMAND). A retailer repeatedly decides how much inventory of an item to order before observing uncertain demand. Storage capacity is limited, unmet demand can be backlogged only up to a prescribed limit, and any demand beyond that limit is lost. The ordering decisions must thus balance ordering costs, holding costs, backlog costs, and lost-sales penalties over time. The state $s \in [l_s, u_s]$ represents the inventory position, with $s > 0$ indicating on-hand stock and $s < 0$ indicating backlogged demand; $l_s < 0$ and $u_s > 0$ bound the backlog and storage capacity, respectively. Each period, the retailer places an order $a \in [0, \bar{a}]$ before observing demand G. We denote by P_G the probability distribution of G, and write $G \sim P_G$. After demand is realized, inventory evolves as $s' = \min\big(\max(s + a - G,\ l_s),\ u_s\big)$. Demand that drives inventory below l_s is partially lost; stock that would exceed u_s is disposed of. The one-period cost is:

$$c(s,a) = c_p a + c_h\,\mathbb{E}[(s')^+] + c_b\,\mathbb{E}[(-s')^+] + c_d\,\mathbb{E}[(s+a-G-u_s)^+] + c_l\,\mathbb{E}[(l_s-s-a+G)^+]. \quad (9)$$

It aggregates purchasing cost $c_p a$, expected holding cost $c_h\mathbb{E}[(s')^+]$ for positive ending inventory, expected backlogging cost $c_b\mathbb{E}[(-s')^+]$ for negative ending inventory, expected disposal cost $c_d\mathbb{E}[(s+a-G-u_s)^+]$ for inventory above capacity, and expected lost-sales cost $c_l\mathbb{E}[(l_s-s-a+G)^+]$ for demand beyond the backlog limit.

We consider an instance of this problem from Lin et al. [14, §6] with discount factor $\gamma = 0.95$, backlog limit $l_s = -4$, storage capacity $u_s = 12$, the maximum order quantity $\bar{a} = 6$, demand $G \sim \mathcal{N}(6, (3.5)^2)$ truncated to $[0, 10]$, purchasing cost $c_p = 20$, holding cost $c_h = 1$, backlogging

cost $c_b = 25$, disposal cost $c_d = 5$, and lost-sales cost $c_l = 3000$. The high lost-sales penalty relative to purchasing cost creates a strong incentive to maintain inventory above the backlog limit. For a quadratic VFA $V(s;\beta) = \beta_0 + \beta_1 s + \beta_2 s^2$, where $\varphi(s) = (s, s^2)$ and $N = 2$, the constraint violation function in (8) becomes $f(\beta; s, a) = \big(c(s,a) - \beta_1\big(s - \gamma\mathbb{E}[s' \mid s,a]\big) - \beta_2\big(s^2 - \gamma\mathbb{E}[(s')^2 \mid s,a]\big)\big)\big/\big(1-\gamma\big)$. Figure 3 shows $f(\beta;\cdot,\cdot)$ for a fixed β. The violation landscape is highly nonconvex over $[-4,12] \times [0,6]$, with curved level sets and substantial variation in local geometry. This nonconvexity comes from the problem primitives: the hinge terms $(\cdot)^+$ in the cost (9) and the clipped transition $s' = \min(\max(s + a - G, l_s), u_s)$. Darker regions indicate smaller values of f and hence more severe ALP constraint violations. The most violated constraints, those with $f(\beta; s, a) < \beta_0$, concentrate in a small dark region and may therefore be missed by any fixed constraint-sampling distribution not adapted to the current coefficient vector. □

Example 6 illustrates the COR challenges. The quadratic basis is natural, but its ability to approximate V^* under the hinge-type costs in (9) is unknown ex ante. The constraints are also challenging: constraint generation repeatedly solves the nonconvex separation problem $\min_{(s,a)}\{f(\beta;s,a) - \beta_0\}$, while constraint sampling requires choosing a sampling distribution that somehow places mass on the most violated regions. The next two subsections make these choices self-adapting.

4.2. Constraint Violation Learning (Construct + Optimize)

The motivation for constraint violation learning (CVL) [14] is that identifying the single most violated constraint, such as the red dot in Figure 3, is stronger than necessary. It suffices for β-updates to receive feedback from state-action pairs with large violations, such as those in the dark regions of Figure 3. This motivates a saddle-point reformulation of the ALP that learns a sampling distribution concentrated on high-violation regions, rather than repeatedly solving a global separation problem. Sampling from this distribution yields informative state-action pairs for the β-updates without global search, so each PSMD iteration requires only sampling and a first-order update instead of solving the separation problem used in constraint generation.

For **construct**, we now develop an optimization model that can represent and learn such distributions directly. We begin with the saddle-point reformulation of the ALP from Lin et al. [14, §4].[2] Since the ALP objective is increasing in the intercept β_0 subject to $\beta_0 \le f(\beta; s, a)$ for all (s,a), at optimality we have $\beta_0 = \min_{(s,a)\in\mathcal{S}\times\mathcal{A}_s} f(\beta; s, a)$. Substituting this into the ALP objective gives

[2] The source paper [14] denotes the basis function weights θ and the constant shift τ. We use $\beta = (\beta_0, \beta_1, \ldots, \beta_N)$ throughout the tutorial for consistency, with β_0 playing the role of τ.

$$F := \max_{\beta \in \mathcal{B}} \min_{(s,a) \in \mathcal{S} \times \mathcal{A}_s} f(\beta; s, a) + \sum_{n=1}^{N} \beta_n \, \mathbb{E}_\nu[\varphi_n(s)], \tag{SD}$$

where $\mathcal{B} \subset \mathbb{R}^N$ is a compact set containing optimal non-intercept ALP solutions $(\beta_1^*, \ldots, \beta_N^*)$. The optimal value F equals the optimal ALP value, so (SD) is equivalent to ALP. The inner problem identifies the single most violated constraint for a given β and is nonconvex in (s, a). This is exactly the separation problem that prevents constraint generation from being broadly applicable.

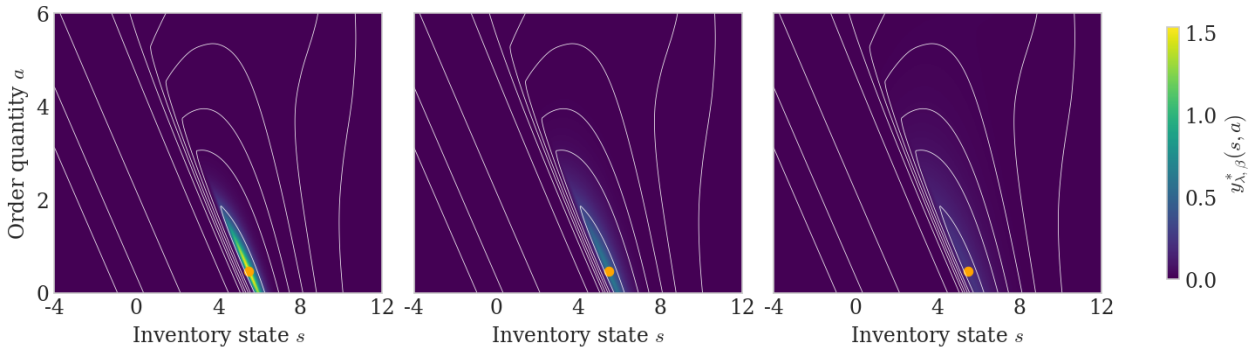
The construct stage converts this nonconvex inner problem into a linear one through *lifting* and *regularization*. Together, these steps replace the search for a single most-violating state-action pair with a sampling distribution over high-violation regions. In the lifting step, the point-valued decision variable $(s, a) \in \mathcal{S} \times \mathcal{A}_s$ is replaced by a density-valued decision variable $y \in \mathcal{Y}$, where $\mathcal{Y}$ is the set of continuous probability densities supported on $\{(s, a) : s \in \mathcal{S},\ a \in \mathcal{A}_s\}$. Thus, instead of evaluating violation at one state-action pair, we average it under y. This replaces $\min_{(s,a) \in \mathcal{S} \times \mathcal{A}_s} f(\beta; s, a)$ with the lifted problem $\inf_{y \in \mathcal{Y}} \mathbb{E}_y[f(\beta; s, a)]$. Lifting preserves the optimal value: an average of f cannot fall below its minimum, so $\mathbb{E}_y[f(\beta; s, a)] \geq \min_{(s,a)} f(\beta; s, a)$; conversely, densities concentrating near a minimizer make the expectation arbitrarily close to the minimum. The benefit is that the objective is linear in y, so the nonconvex search over state-action pairs is replaced by a linear optimization over densities. However, this linear program is infinite-dimensional, and its optimal solution can place most of its mass on increasingly small regions where constraint violation is largest. In effect, the density can collapse onto the most violated state-action pair, recovering the original separation problem. Such concentration is undesirable because it undermines the goal of learning and sampling from broader violating regions.

In the regularization step, we prevent this degeneracy by penalizing overly concentrated densities, much as regularization controls model complexity in regression and machine learning. Lin et al. [14] use a Kullback-Leibler penalty $\lambda D_{\mathrm{KL}}(y \,\|\, p_u)$, where p_u is the uniform density on $\mathcal{S} \times \mathcal{A}_s$ and $\lambda > 0$. This penalty measures how far y is from the uniform distribution: as y concentrates on smaller regions, its divergence from p_u grows, so regularization promotes a smoother distribution over highly violated regions. The resulting regularized saddle-point formulation is

$$F(\lambda) := \max_{\beta \in \mathcal{B}} \min_{y \in \mathcal{Y}} \mathbb{E}_y[f(\beta; s, a)] \ + \ \lambda \, D_{\mathrm{KL}}(y \,\|\, p_u) \ + \ \sum_{n=1}^{N} \beta_n \, \mathbb{E}_\nu[\varphi_n(s)]. \tag{10}$$

Since $D_{\mathrm{KL}}(y \,\|\, p_u) \geq 0$, regularization can only increase the inner-minimization value, so $F(\lambda) \geq F$ for all $\lambda > 0$. As $\lambda \to 0$, the regularization vanishes and the original saddle-point problem

Figure 4 **Gibbs density $y^*_{\lambda,\beta}$ for inventory Example 6 with $\lambda = 50$ (left), $\lambda = 100$ (middle), and $\lambda = 200$ (right).**



is recovered, implying $F(\lambda) \to F$. For fixed β, first-order optimality conditions for the inner minimization yield the following closed-form Gibbs distribution in $\mathcal{Y}$ [14, Proposition 2]:

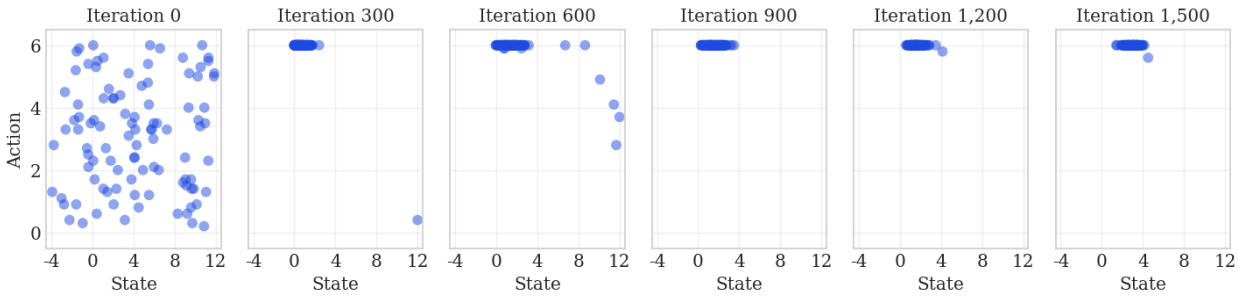
$$y^*_{\lambda,\beta}(s,a) = \frac{\exp(-f(\beta;\, s,a)/\lambda)}{\int_{\mathcal{S}\times\mathcal{A}_s} \exp(-f(\beta;\, s',a')/\lambda)\,\mathrm{d}(s',a')}. \tag{11}$$

Equation (11) explicitly characterizes the optimal density. The numerator weights each state-action pair by $f(\beta; s, a)$, while the denominator normalizes these weights to define a valid probability density. Because $\exp(-f/\lambda)$ decreases in f, the Gibbs density places more mass where the violation $-f$ is largest, exactly the regions CVL targets. Although the normalization constant may not be available in closed form, the density can still be sampled without explicitly computing it, for example, using the Metropolis-Hastings algorithm. Thus, the construct stage converts a difficult nonconvex search into sampling from a distribution concentrated on highly violated regions. Here, λ controls how strongly the Gibbs density concentrates. As $\lambda \to 0$, $y^*_{\lambda,\beta}$ collapses toward the most violated constraint, approaching the original nonconvex inner minimization. As λ increases, the density spreads mass over nearby highly violated constraints, easing sampling but increasing the smoothing bias $F(\lambda) - F$. Figure 4 illustrates this effect for Example 6, with β fixed and $\lambda \in \{50, 100, 200\}$. Smaller λ produces a sharp peak near the most violated region (orange dot), while larger λ spreads mass more broadly over nearby high-violation regions. Overall, the construct stage yields a model whose inner minimizer is a closed-form density that can be sampled using standard methods.

For **optimize**, we must solve (10), where β maximizes the objective, and the density y minimizes it. Here, a primal-dual first-order method is natural, but two issues arise. First, the updates require expectations under the current density y, which are generally intractable over the continuous state–action space. Following Lin et al. [14], PSMD replaces these expectations with unbiased estimates: at each iteration, a sample $(\hat{s}, \hat{a}) \sim y$ provides the first-order information for updating β. Second, the variables have different geometry: Euclidean proximity is natural for $\beta \in \mathbb{R}^N$, while KL divergence

is natural for the density y. Thus, PSMD uses a Euclidean primal update for β and a KL-based mirror update for y. It initializes $\beta = \mathbf{0}$ and $y = p_u$. At each iteration, the primal update uses a sample from y to update the VFA parameter β; the dual update then uses the updated violation landscape to revise y, producing the sampling distribution for the next iteration. At termination, the output policy is the greedy policy $\pi_g(\cdot; V(\bar{\beta}))$, with the argmax in §2.2 replaced by an argmin for the cost setting. Here, $\bar{\beta}$ is the terminal primal solution. Please see §5 of Lin et al. [14] for the details of the primal-dual updates, including the update of λ, and convergence results, and §6 for implementation details.

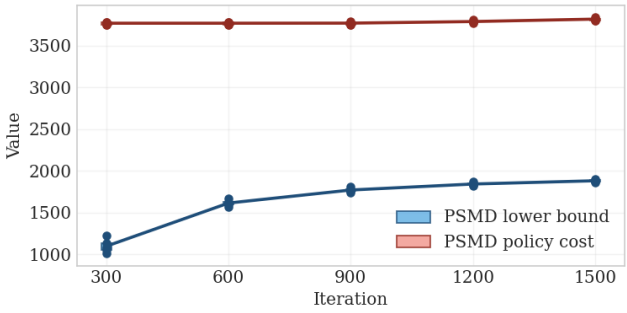
Figure 5 Samples from y^t across PSMD iterations for Example 6.



The interaction between the primal and dual updates makes PSMD self-adapting. As y concentrates on regions with large constraint violations, the primal update is increasingly driven by samples from those regions. The resulting change in β reshapes the violation landscape, updating the Gibbs density and redirecting future samples. Meanwhile, PSMD reduces the regularization parameter λ across iterations, sharpening the Gibbs density and allowing the algorithm to focus more selectively on dominant violation regions. Figure 5 illustrates this feedback mechanism for Example 6. Initially, the density is nearly uniform and samples are spread broadly across the state-action space. As the algorithm progresses, the density concentrates near high-violation regions, so future samples and updates increasingly focus there. In the inventory example, samples concentrate on small positive inventory levels and the maximum replenishment action $a = \bar{a}$. When inventory is low, shortage and lost-sales costs dominate holding costs, making aggressive replenishment desirable. These state-action pairs correspond to binding ALP constraints, so the Gibbs density $\propto \exp(-f/\lambda)$ increasingly concentrates on them. Future updates are therefore driven by the constraints closest to violation and most important to the current approximation.

Certified optimality gap. A practical advantage of (10) is that it yields a computable solution-quality certificate. As in constraint generation, simulating the greedy policy $\pi_g(\cdot; V(\bar{\beta}))$ gives an upper bound on the optimal cost. The saddle-point formulation also provides a lower bound

Figure 6 Lower bound and simulated policy cost over PSMD iterations for Example 6.



through weak duality. Without PSMD, computing this bound exactly would require solving the original nonconvex separation problem, but the density y in PSMD can be used to compute a certified lower bound $\ell(\bar{\beta})$ satisfying $\ell(\bar{\beta}) \leq F$ (see §5.2 of [14] and §EC.4.1 of [22]). Thus, PSMD produces both upper and lower bounds during its execution. Moreover, Lin et al. [14, Theorem 1] show that the primal-dual gap of (10) falls below ϵ with high probability after $O(1/\epsilon^2)$ iterations.

This convergence result assumes that the MDP value function is spanned by the chosen basis functions. Thus, unlike the optimization error, which PSMD is guaranteed to reduce, the representation error has no such guarantee: it depends on the expressive power of the VFA class and must be assessed empirically. Figure 6 illustrates this distinction for Example 6. As PSMD progresses, the lower bound increases steadily while the simulated policy cost changes little, shrinking the certified optimality gap. The remaining gap, however, remains substantial, suggesting that the dominant error is no longer optimization but approximation: the quadratic VFA is not expressive enough to accurately represent the optimal value function. In Section 4.3, we show that this gap can be reduced by enriching the VFA class, vanishing once the approximation space is sufficiently expressive.

Summary. In the COR framework, CVL adapts the construct and optimize stages. *Construct* replaces nonconvex separation with a lifted density formulation and uses KL regularization to obtain a sampleable Gibbs density. *Optimize* applies PSMD, which updates both the VFA coefficients and the sampling density using samples from the current Gibbs-type distribution. *Refine* is implicit: the density y adapts during the solve, redirecting attention toward violated regions, while the basis functions and VFA class remain fixed.

Limitations. CVL is designed for settings where separation is difficult. When exact separation is tractable, constraint generation is preferable because it directly targets the worst violation and may progress faster than sampling-based methods. CVL is useful when separation is intractable while sampling from the Gibbs density is feasible, replacing global optimization with targeted sampling. Its main limitation is that the computational burden shifts from separation to sampling:

as the state-action dimension grows, the Gibbs density may concentrate on unknown, multimodal regions, making exploration harder and reducing sample quality. This does not change the $O(1/\epsilon^2)$ convergence guarantee, but it can increase gradient variance and slow practical convergence. Thus, in high dimensions, exploiting problem structure is often essential. Even then, PSMD per-iteration cost remains attractive: each step scales linearly with the number of basis functions N and avoids enumerating the state-action space. Thus, dimension enters mainly through Gibbs sampling, not the per-step algebra. By contrast, standard ALP faces dimension more directly: constraint generation must solve a global nonconvex separation problem over the high-dimensional continuum at each iteration, while constraint sampling must cover that continuum well enough to retain accuracy.

4.3. Self-Guided Approximate Linear Programs (Construct + Optimize + Refine)

CVL treats the basis-function class as fixed: if the chosen features are rich enough to approximate V^*, CVL can solve the corresponding ALP and report a certified gap. However, the right features are rarely known in advance. In inventory Example 6, Figure 6 shows that a substantial certified optimality gap remains even after the ALP is solved accurately, suggesting that the quadratic features 1, s, and s^2 are not expressive enough. Manually improving the basis would require trial and error, exactly the burden the COR cycle aims to reduce. Self-guided ALPs address this limitation by making the approximation architecture adaptive: the method samples basis functions from a prescribed random-feature family, solves the resulting ALP, and enriches the approximation when the policy and certificate indicate that more expressive power is needed. For example, in the inventory problem, the quadratic VFA can be augmented with random Fourier features $\varphi_n(s) = \cos(\theta_{n,0} + \theta_{n,1}s)$, where $\theta_n = (\theta_{n,0}, \theta_{n,1})$ is sampled from a normal distribution. This expands the approximation class from $V(s;\beta) = \beta_0 + \beta_1 s + \beta_2 s^2$ to $V(s;\beta) = \beta_0 + \beta_1 s + \beta_2 s^2 + \sum_{n=1}^{N} \beta_{n+2}\varphi_n(s)$, allowing the value function to capture richer nonlinear behavior.

In the COR view, self-guided ALP is iterative. In the construct stage, the user selects a random-feature family, and the algorithm samples basis functions from it to form an initial VFA class. In the optimize stage, the resulting ALP is solved using the standard ALP methods in §2.2, chosen according to the problem structure. In the refine stage, the method adds new random features in batches and imposes guiding constraints built from earlier VFAs. These constraints require the richer VFA with larger N to improve on the previous VFA with smaller N, directing attention to regions where the earlier approximation was weakest. Thus, self-guided ALP avoids requiring a large hand-designed basis upfront; it repeatedly constructs, solves, and refines the approximation, adapting both the basis functions and the state-space emphasis across rounds.

For **construct**, self-guided ALPs sample N parameter vectors $\theta_n \in \mathbb{R}^{d_S+1}$ independently from a prescribed distribution ϑ[3] and define $\varphi_n(s) = \phi(\theta_n^\top(1,s))$, where $\phi : \mathbb{R} \to \mathbb{R}$ is a fixed activation function. The random Fourier construction above corresponds to $\phi(\cdot) = \cos(\cdot)$ with ϑ chosen as a multivariate normal distribution. With suitable choices of (ϕ, ϑ), the resulting random feature family is universal such that as N grows, linear combinations $V(s;\beta) := \sum_{n=1}^{N} \beta_n \varphi_n(s)$ can approximate any continuous value function, including V^*, arbitrarily well with high probability [28]. Examples of universal random-feature constructions are summarized in Table 1 of Pakiman et al. [22].

Substituting the random-feature VFA into the ALP, together with a uniform state-relevance distribution, gives the feature-based approximate linear program [22, §3], FALP_N:

$$\text{LB}^{\text{FA}}_{N,\text{unif}} := \max_{\beta \in \mathbb{R}^{N+1}} \left\{ \mathbb{E}_{\text{unif}}[V(s;\beta)] \quad \text{s.t.} \quad \beta_0 \le f(\beta;s,a),\ \forall (s,a) \in \mathcal{S} \times \mathcal{A}_s \right\}.$$

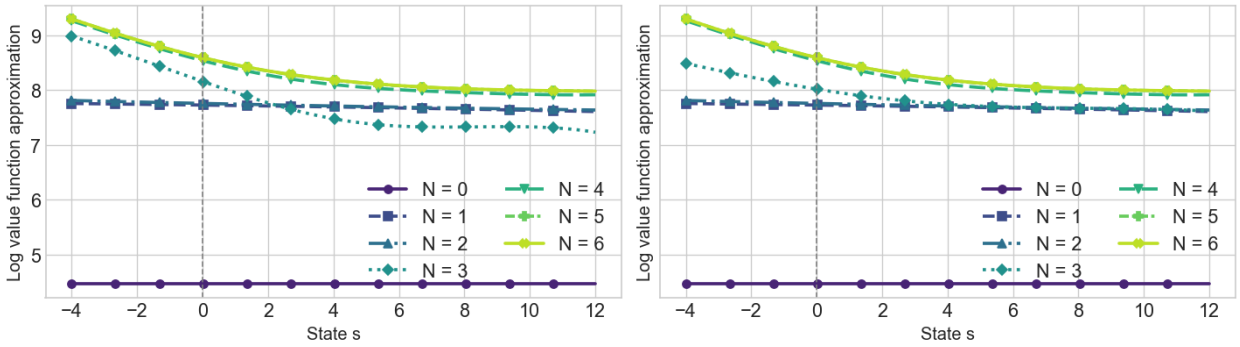
FALP_N is a randomized linear model because its columns depend on the realizations of $\theta_n \sim \vartheta$. Let β_N^{FA} denote an FALP_N optimal solution. Under suitable regularity conditions on V^* and a universality assumption, Pakiman et al. [22, Theorem 1] establish that $\|V^* - V(\cdot;\beta_N^{\text{FA}})\|_{1,\text{unif}} := \mathbb{E}_{\text{unif}}\left[\, |V^*(s) - V(s;\beta_N^{\text{FA}})| \,\right]$ converges to zero at rate $1/\sqrt{N}$ with high probability: a *representation*-error guarantee in the limit of many features, complementing the optimization guarantee of PSMD. The user controls approximation quality by choosing N, with larger N yielding a richer function class at the cost of a larger optimization problem.

For **optimize**, FALP_N remains a semi-infinite LP in β, so the standard solution routes from §2.2 apply. Constraint sampling is straightforward but inherits the limitations discussed there. Constraint generation requires tailoring the separation problem to the random-feature family (ϕ, ϑ) and the MDP primitives. When the activation has structure, separation may admit a tractable mixed-integer reformulation; examples include piecewise-constant activations for random stumps and piecewise-linear activations for random ReLU features (see Table 1 of [22]). For example, Pakiman and Nadarajah [21] develop such a reformulation for a generalized joint replenishment problem. This stage generally requires problem-specific work, and combining FALP_N with CVL is a plausible direction. The left panel of Figure 7 shows the sequence of FALP_N VFAs, for $N = 0, 1, \ldots, 6$, obtained by constraint sampling in Example 6. The VFA is constant at $N = 0$ and becomes progressively more flexible as random features are added, allowing it to capture richer structure.

For **refine**, solving a sequence of FALP_N models with increasing N does not by itself transfer information across solves. This is wasteful because every feasible ALP solution is a state-wise

[3] In Pakiman et al. [22], we use notation ρ rather than ϑ for the random features sampling distribution.

Figure 7 **Sequences FALP$_N$ VFA (left) and FALP$_N^{SG}$ VFA (right) for inventory Example 6.**



lower bound on V^*. A previously computed VFA may already approximate V^* well in parts of the state space, so there is little value in rediscovering those regions. Self-guided ALP preserves this information through a guiding constraint that requires each new VFA to lie above the previous one at every state [22, §4]. Thus, each refinement builds on what has already been learned and directs new approximation power toward the remaining gaps. Self-guided ALP adds random features in batches of size B. At round q, with $N = qB$ random features, it solves the feature-guided LP FALP$_N^{SG}$:

$$\max_{\beta\in\mathbb{R}^{N+1}} \left\{ \mathbb{E}_{\text{unif}}[V(s;\beta)] \;\text{ s.t. }\; \beta_0 \le f(\beta;s,a),\ \forall (s,a)\in\mathcal{S}\times\mathcal{A}_s,\ \ V(s;\beta)\ge V(s;\beta^{SG}_{N-B}),\ \forall s\in\mathcal{S} \right\}, \quad (12)$$

where β^{SG}_{N-B} denotes the optimal solution of FALP$^{SG}_{N-B}$ from the previous iteration, with the convention $V(s;\beta^{SG}_{-B}) \equiv -\infty$ at $N = B$ so that the guiding constraint is redundant at the first iteration. Such a model is well defined: appending B zeros to β^{SG}_{N-B} gives a feasible point at which the guiding constraint holds with equality. Since both VFAs are state-wise lower bounds on V^*, the constraint pushes the sequence of VFAs upward toward V^* across the entire state space, not just on average. The right panel of Figure 7 shows the same progression for the self-guided models in Example 6. Unlike the FALP$_N$ VFAs in the left panel, the FALP$_N^{SG}$ VFAs increase state by state as N grows. This is exactly the effect of the guiding constraint. By contrast, the unguided FALP$_N$ sequence need not be monotone; for example, the FALP$_2$ VFA lies above the FALP$_4$ VFA for all states $s \ge 2$.

The guiding constraint has a more fundamental role than simply making the VFA sequence monotone. By requiring each new VFA to dominate the previous one state by state, it encourages refinement effort to be concentrated in regions where the current approximation is hardest to improve. This intuition can be made precise through a dual interpretation. Pakiman et al. [22, §4.2] show that the dual multipliers associated with the guiding constraint induce an effective state-relevance distribution ν'. Relative to the original uniform distribution, ν' places greater weight on states where the previous VFA is tight, thereby directing subsequent refinement toward regions that are most informative for improving the approximation.

Figure 8 **Sequences of lower bounds and upper bounds under FALP_N (left) and FALP_N^{SG} (right).**

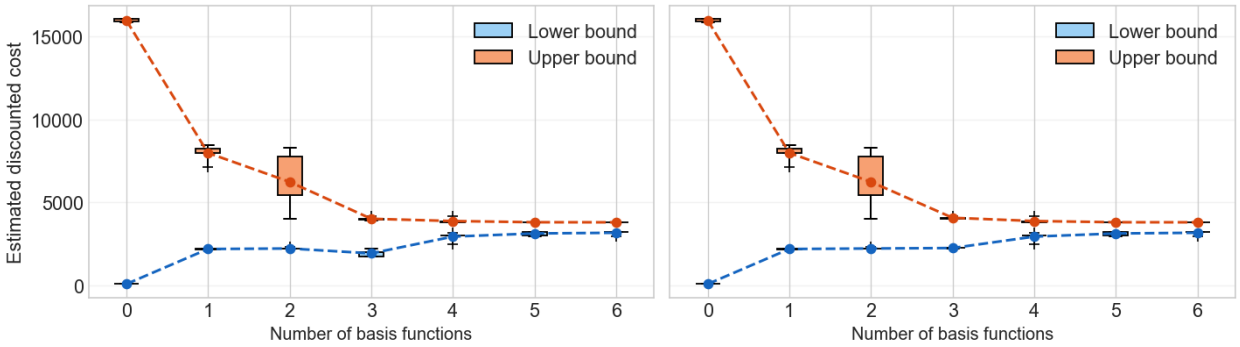


Figure 8 compares the bounds obtained from FALP_N and FALP_N^{SG} for Example 6. Both methods substantially improve the simulated policy cost and the certified lower bound as random features are added. The guiding constraint has a modest effect in this one-dimensional example, but its impact can be much larger in richer instances (see, e.g., Table 2 of Pakiman et al. [22]). The certified lower bound in this figure is computed using the CVL lower-bound estimator, without running the PSMD primal-dual updates (see, e.g., EC.4 of Pakiman et al. [22]). Comparing the optimality gaps in Figure 8 with those from PSMD in Figure 6 illustrates the value of refinement: expanding the basis with random features and adding guiding constraints can produce stronger policies and tighter certificates.

Summary. In the COR framework, self-guided ALP proceeds in three steps. *Construct* samples N random basis functions from a prescribed family (ϕ, ϑ) and forms FALP_N. *Optimize* solves FALP_N using constraint generation, constraint sampling, or potentially CVL, depending on the random-feature class and MDP structure. *Refine* adds features in batches and imposes the guiding constraint $V(s;\beta) \geq V(s;\beta_{N-B}^{SG})$, so each new VFA builds on the previous one rather than starting from scratch. Compared with CVL, adaptation occurs in a different place: CVL adapts the dual density y over the state-action space to focus optimization on violated constraints, while self-guided ALP adapts the basis class and, implicitly through the guiding-constraint duals, the state-relevance distribution over the state space. The two methods are complementary: CVL automates optimization for a fixed basis, whereas self-guided ALP automates construction and refinement given a solution method for each FALP_N.

Limitations. Self-guided ALP reduces the need to manually design basis functions and state-relevance distributions by using random features and guiding constraints in an application-agnostic way. This generality, however, does not remove high-dimensional difficulty. As the state dimension grows, more random features may be needed to approximate V^* well, and each refinement round still requires solving an ALP with constraints over a high-dimensional continuous space. In very

high-dimensional problems with exploitable structure, tailored basis functions and a carefully chosen state-relevance distribution may outperform self-guided ALP precisely because they use problem knowledge. Even then, self-guided ALP remains a useful benchmark for assessing the value of customized approximations.

Getting started with code. Code accompanying this section is provided in the tutorial repository under the folder `continuous-mdp`[4]. This folder contains three notebooks, organized in the order of §4.2 and §4.3, that illustrate the methods on the inventory instance of Example 6.

5. Emerging Frontiers

The methods in this tutorial are enabled by two waves of solver maturation. Mathematical programming solvers power the weakly coupled methods of §3, and first-order methods power the continuous-state methods of §4. The COR cycle in each case is reorganized around what the underlying solver does well, and the user is freed from design choices that those solvers can absorb. We close by noting where these solver paradigms continue to push outward, and where two further waves are beginning to take shape.

The COR cycle has more reach than this tutorial has exercised. Two recent directions extend it into parts of the formulation that Sections 3 and 4 hold fixed. The methods in §4 fix the temporal architecture of the approximation, coupling decisions across all stages through a single joint LP whose computational cost grows with the horizon for finite-horizon MDPs with high-dimensional exogenous uncertainty. Removing this coupling at the construct stage and pairing it with a parallel block-coordinate descent at the optimize stage delivers bounds whose complexity is independent of the horizon [33]. The methods in both sections also assume that the MDP is given exactly, with no ambiguity in its primitives. In dynamic pricing under demand model ambiguity, allowing refine to update the ambiguity set as sales data arrives, and feeding the contracted set back into construct, makes the level of risk aversion endogenous to the data stream rather than a fixed input [20]. Both directions adapt parts of the COR cycle that this tutorial holds fixed, while leaning on the same mathematical programming and first-order solver machinery used throughout.

A third wave is beginning to take shape in foundation models and large language models, which are positioned to add a layer of judgment around the existing solvers rather than replace them. They can read a problem description, including code, mathematical text, and unstructured data, and propose a model representation, which makes the construct stage itself open to automation.

[4] The source files are included with the review package. A public GitHub repository will be made available after the review process.

They can read intermediate output, including partial solutions and dual prices, and decide what to refine next, which opens the refine stage to richer signals than the residuals and reduced costs that classical solvers expose. They can also act as approximate oracles for subproblems whose exact solution is the bottleneck of methods like delayed allocation, in the spirit of the approximate-oracle results in §3.3 but for oracles that need not be specified in closed form. How to use these models with bounds and guarantees, rather than ad hoc prompting, is an open question.

A fourth wave acts on the optimize stage directly. Quantum computing has matured to the point where small QUBO and SDP instances can be solved on physical hardware, and INFORMS TutORials has covered both the QUBO route on quantum computers [16] and the SDP route via quantum information [32]. For MDP approximation, the relevant question is which optimize-stage subproblems within the COR cycle (e.g., primal-dual updates for constraint violation learning), admit reformulations that quantum methods can accelerate as the hardware scales. The construct and refine stages of the cycle would remain classical, but a quantum optimize stage would change which problem sizes are reachable and, in turn, how aggressively self-adaptation can be pursued.

References

[1] Adelman D, Mersereau A, Pakiman P (2024) Dynamic optimization of workforce talent. *Available at SSRN 4964890* .

[2] Adelman D, Mersereau AJ (2008) Relaxations of weakly coupled stochastic dynamic programs. *Operations Research* 56(3):712–727.

[3] Agrawal S (2019) Recent advances in multiarmed bandits for sequential decision making. *INFORMS TutORials in Operations Research*, 167–188.

[4] Bertsekas DP (2025) Neuro-dynamic programming. *Encyclopedia of Optimization*, 1–6 (Springer).

[5] Bertsimas D, Mersereau AJ (2007) A learning approach for interactive marketing to a customer segment. *Operations Research* 55(6):1120–1135.

[6] Bertsimas D, Mišić VV (2016) Decomposable markov decision processes: A fluid optimization approach. *Operations Research* 64(6):1537–1555.

[7] Brown DB, Smith JE (2020) Index policies and performance bounds for dynamic selection problems. *Management Science* 66(7):3029–3050.

[8] Brown DB, Zhang J (2025) Fluid policies, reoptimization, and performance guarantees in dynamic resource allocation. *Operations Research* 73(2):1029–1045.

[9] Caro F, Gallien J (2007) Dynamic assortment with demand learning for seasonal consumer goods. *Management Science* 53(2):276–292.

[10] D'Aeth JC, Ghosal S, Grimm F, Haw D, Koca E, Lau K, Liu H, Moret S, Rizmie D, Smith PC, et al. (2023) Optimal hospital care scheduling during the SARS-CoV-2 pandemic. *Management Science* .

[11] de Farias DP, Van Roy B (2003) The linear programming approach to approximate dynamic programming. *Operations Research* 51(6):850–865.

[12] de Jonge B, Scarf PA (2020) A review on maintenance optimization. *European Journal of Operational Research* 285(3):805–824.

[13] Fu MC (2017) Markov decision processes, AlphaGo, and Monte Carlo tree search: Back to the future. *INFORMS TutORials in Operations Research*, 68–88.

[14] Lin Q, Nadarajah S, Soheili N (2020) Revisiting approximate linear programming: Constraint-violation learning with applications to inventory control and energy storage. *Management Science* 66(4).

[15] Mannor S, Xu H (2019) Data-driven methods for Markov decision problems with parameter uncertainty. *INFORMS TutORials in Operations Research*, 101–129.

[16] Mazumder AR, Tayur S (2025) Five starter problems: Solving quadratic unconstrained binary optimization models on quantum computers. *INFORMS TutORials in Operations Research* (INFORMS).

[17] Nadarajah S, Cire AA (2025) Self-adapting network relaxations for weakly coupled Markov decision processes. *Management Science* 71(2).

[18] Nadarajah S, Cire AA (2026) Delayed allocation in marginalized flow models for weakly coupled Markov decision processes. *SSRN* URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6127326.

[19] Ning J (2017) Easy affine Markov decision processes: Properties and applications. *INFORMS TutORials in Operations Research*, 28–47.

[20] Pakiman P, Chen B, Nadarajah S, Jasin S (2025) Adaptive risk mitigation in demand learning. *arXiv preprint arXiv:2011.10690* URL https://arxiv.org/abs/2011.10690.

[21] Pakiman P, Nadarajah S (2024) Back to the future: Revisiting a pioneering approximation of average cost markov decision processes using a multi-shot perspective. *SSRN* URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4938885.

[22] Pakiman P, Nadarajah S, Soheili N, Lin Q (2025) Self-guided approximate linear programs: Randomized multi-shot approximation of discounted Markov decision processes. *Management Science* 71(4).

[23] Powell WB (2011) *Approximate Dynamic Programming: Solving the Curses of Dimensionality* (John Wiley & Sons).

[24] Powell WB (2014) Clearing the jungle of stochastic optimization. *INFORMS TutORials in Operations Research*, 109–137.

[25] Powell WB (2016) A unified framework for optimization under uncertainty. *INFORMS TutORials in Operations Research*, 45–83.

[26] Powell WB, Topaloglu H (2006) Approximate dynamic programming for large-scale resource allocation problems. *INFORMS TutORials in Operations Research*, 123–147.

[27] Puterman ML (2014) *Markov Decision Processes: Discrete Stochastic Dynamic Programming* (John Wiley & Sons).

[28] Rahimi A, Recht B (2008) Uniform approximation of functions with random bases. *2008 46th annual allerton conference on communication, control, and computing*, 555–561 (IEEE).

[29] Schweitzer PJ, Seidmann A (1985) Generalized polynomial approximations in markovian decision processes. *Journal of Mathematical Analysis and Applications* 110(2):568–582.

[30] Scully Z, Terenin A (2025) The Gittins index: A design principle for decision making under uncertainty. *INFORMS TutORials in Operations Research*.

[31] Sellmann M (2015) Meta-algorithms. *INFORMS TutORials in Operations Research*, 47–55.

[32] Siddhu V, Tayur S (2022) Five starter pieces: Quantum information science via semidefinite programs. *INFORMS TutORials in Operations Research* (INFORMS).

[33] Soheili N, Nadarajah S, Yang B (2026) Weakly time-coupled approximation of Markov decision processes. *arXiv preprint arXiv:2603.12636* URL https://arxiv.org/abs/2603.12636.

[34] Sutton RS, Barto AG (2018) *Reinforcement Learning: An Introduction* (MIT Press), 2nd edition.

[35] Wu T, Zurek M, Chen Y, Wang W, Xie Q (2025) Lyapunov-based sample complexity analysis for weakly-coupled MDPs. *NeurIPS 2025 Workshop MLxOR: Mathematical Foundations and Operational Integration of Machine Learning for Uncertainty-Aware Decision-Making*, URL https://openreview.net/forum?id=npV2sxCM7P.

[36] Zayas-Caban G, Jasin S, Wang G (2019) An asymptotically optimal heuristic for general nonstationary finite-horizon restless multi-armed, multi-action bandits. *Advances in Applied Probability* 51(3):745–772.